ĐẠI HỌC QUỐC GIA TPHCM
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO ĐỒ ÁN 1

TRÒ CHƠI BLOXORZ VÀ CÁC THUẬT TOÁN TÌM KIẾM

Môn học: Cơ sở Trí tuệ Nhân tạo
Lớp: CQ2024/23

Giảng viên hướng dẫn
TS. Bùi Duy Đăng
Nguyễn Ngọc Đức
Nguyễn Thanh Tình

Sinh viên thực hiện

MSSV	Họ và tên
24120238	Phạm Anh Tuấn
24120240	Võ Lân Tuấn
24120249	Võ Nguyễn Việt Hoàng
24120290	Lê Tấn Dũng
24120470	Dương Phan Minh Trí

Mục lục

1 Giới thiệu và phạm vi **1**

1.1 Mục tiêu dự án . . . 1

1.2 Phạm vi thực hiện . . . 1

2 Phân công công việc và đóng góp **2**

3 Phát biểu bài toán và mô hình hình thức **2**

3.1 Mục tiêu và luật trò chơi . . . 2

3.2 Đặc tả JSON và ràng buộc . . . 3

3.3 Ví dụ minh họa . . . 3

3.4 Mô hình trạng thái . . . 4

3.5 Thứ tự hiệu ứng trong một lượt . . . 5

3.6 Hàm chi phí và đích . . . 6

3.7 So sánh trạng thái lặp . . . 7

4 Thiết kế hệ thống và các thuật toán tìm kiếm **8**

4.1 Kiến trúc tổng thể và luồng dữ liệu . . . 8

4.2 Chơi thủ công, điều khiển và quản lý trạng thái . . . 10

4.2.1 Vòng đời ván chơi và tư thế khối . . . 10

4.2.2 Hoàn tác (Undo) và đặt lại (Reset) . . . 12

4.3 Tích hợp solver, luồng nền và tạm dừng tìm kiếm . . . 13

4.3.1 Lý do dùng luồng nền . . . 13

4.3.2 Tạm dừng và tiếp tục ngay trong lúc tìm kiếm . . . 13

4.3.3 Thời gian tạm dừng và timeout . . . 14

4.3.4 Loại bỏ kết quả cũ khi Reset hoặc đổi màn . . . 15

4.4 Hoạt ảnh 3D và đồng bộ replay . . . 15

4.4.1 Chuyển động lăn khối . . . 15

4.4.2 Rơi khỏi mép sàn . . . 15

4.4.3 Hiệu ứng đặc biệt . . . 16

4.4.4 Replay lời giải . . . 17

4.5 Breadth-First Search (BFS) . . . 17

4.6 Depth-First Search (DFS) . . . 18
4.7 Uniform-Cost Search (UCS) . . . 19
4.8 A* . . . 20
4.9 So sánh các thuật toán . . . 23

5 Phương pháp thực nghiệm 24
5.1 Nguồn dữ liệu và độ bao phủ tính năng . . . 24
5.2 Thiết lập thí nghiệm . . . 26
5.3 Quy trình và phép đo . . . 27

6 Kết quả và thảo luận 29
6.1 Thời gian tìm kiếm . . . 29
6.2 Peak memory . . . 30
6.3 Expanded nodes . . . 31
6.4 Độ dài lời giải và chi phí . . . 31
6.5 Checker verdict . . . 32
6.6 Thảo luận kết quả . . . 33

7 Hạn chế và hướng phát triển 34
7.1 Hạn chế hiện tại . . . 34
7.2 Hướng phát triển . . . 35
7.3 Ghi nhận nguồn và quyền sử dụng . . . 35

Tài liệu tham khảo 36

A Cài đặt và thực thi 37
A.1 Báo cáo LaTeX . . . 37
A.2 Engine và solver . . . 37
A.3 Hướng dẫn vận hành giao diện . . . 38

B Lệnh benchmark và kết quả chi tiết 39
B.1 Lệnh chạy . . . 39
B.2 Kết quả theo stage . . . 39

C Ma trận kiểm thử giao diện và engine **46**

C.1 Ma trận 19 ca kiểm thử 46

C.2 Trace đối chiếu ba bên 48

D Video demo và mã nguồn **48**

E Ghi nhận hỗ trợ AI **49**

F Tài nguyên và thư viện **49**

Danh sách bảng

1 Bảng phân công và đánh giá giữa các thành viên. 2
2 Các luật và cơ chế chính của Bloxorz 2
3 Các trường chính của JSON đầu vào 3
4 Các ví dụ minh họa cơ chế trò chơi 4
5 Thứ tự xử lý một lượt 5
6 Độ phức tạp của A* 23
7 So sánh lý thuyết các thuật toán tìm kiếm 23
8 Độ phức tạp lý thuyết của các thuật toán 24
9 Phân nhóm stage original theo cơ chế ưu tiên 25
10 Môi trường phần cứng và phần mềm của thí nghiệm 26
11 Định nghĩa và phạm vi các đại lượng đo 27
12 Thời gian tìm kiếm minh họa theo stage: median (IQR), ms 29
13 Peak memory minh họa theo stage: median (IQR), MiB 30
14 Expanded nodes minh họa theo stage: median (IQR), trạng thái 31
15 Độ dài lời giải L / weighted cost C_U theo stage (median qua năm lượt) 32
16 Tổng hợp checker verdict và xác minh kết quả 33
17 Thời gian giải theo stage: median (IQR), ms 39
18 Peak memory theo stage: median (IQR), MiB 41
19 Expanded nodes theo stage: median (IQR), trạng thái 42
20 Độ dài lời giải theo stage: median (IQR), action 43
21 Weighted cost C_U theo stage: median (IQR) 45
22 Ma trận kiểm thử: kịch bản và kết quả (tất cả `PASSED`) 46

Danh sách hình vẽ

1 Kiến trúc tổng thể và luồng dữ liệu giữa dữ liệu màn, bộ giải, checker và giao diện. 9
2 Bảng điều khiển chọn một trong bốn thuật toán BFS, DFS, UCS và A*. 10
3 Ba trạng thái GameStatus của engine; SPLIT và FALLING chỉ được chú thích ngoài đồ thị. 11
4 Khối đã tách thành hai cube; cube đang điều khiển có màu đậm hơn. 12
5 Giao diện 3D khi chơi thủ công màn 01. 13
6 Khối đi quá mép và bị rơi; có hoạt ảnh rơi và màn hình thử lại. 16
7 Hiệu ứng chiến thắng khi khối đứng thẳng lọt vào ô đích. 17

1 Giới thiệu và phạm vi

Bloxorz là trò chơi giải đố, trong đó người chơi điều khiển một khối hộp kích thước $1\times1\times2$ lăn trên bàn chơi để đứng thẳng và lọt vào ô đích. Người chơi cần tránh rơi khỏi bàn, đồng thời xử lý các ô đặc biệt như ô dễ vỡ, công tắc điều khiển cầu và ô tách khối. Trò chơi đòi hỏi khả năng suy luận không gian và lập kế hoạch di chuyển, phù hợp để nghiên cứu các thuật toán tìm kiếm trong trí tuệ nhân tạo[2].

1.1 Mục tiêu dự án

Mục tiêu của dự án là xây dựng chương trình giải Bloxorz bằng bốn thuật toán BFS, DFS, UCS và A*, đồng thời so sánh thời gian thực thi, bộ nhớ sử dụng, số nút mở rộng và chất lượng lời giải[8]. Dự án cũng cung cấp giao diện 3D hỗ trợ chơi thủ công và phát lại lời giải, giúp minh họa trực quan kết quả tìm kiếm[1].

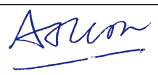
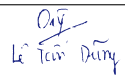
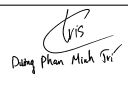
1.2 Phạm vi thực hiện

Dự án triển khai trên 33 màn chơi gốc và 6 màn mẫu, hỗ trợ ô dễ vỡ, cầu, công tắc và tách ghép khối. Dự án chưa bao gồm chức năng tự sinh màn chơi.

Báo cáo trình bày dữ liệu đầu vào, biểu diễn trạng thái và luật chuyển tại Mục 3, bộ dữ liệu và quy trình thực nghiệm tại Mục 5.2, cùng phần mô tả thuật toán và phân tích kết quả.

Mã nguồn và video demo. Mã nguồn trên GitHub; video demo.

2 Phân công công việc và đóng góp

Thành viên	MSSV	Phạm vi phụ trách	Đóng góp	Chữ ký
Phạm Anh Tuấn	24120238	Triển khai và viết báo cáo BFS, DFS.	100%	
Võ Lân Tuấn	24120240	Triển khai thuật toán A*, mô hình hoá bài toán và tổng hợp thực nghiệm.	100%	
Võ Nguyễn Việt Hoàng	24120249	Xây dựng GUI, kiểm thử tương tác và demo.	100%	
Lê Tấn Dũng	24120290	Triển khai và viết báo cáo UCS.	100%	
Dương Phan Minh Trí	24120470	Xây dựng GUI, kiểm thử tương tác và demo.	100%	

Bảng 1: Bảng phân công và đánh giá giữa các thành viên.

3 Phát biểu bài toán và mô hình hình thức

3.1 Mục tiêu và luật trò chơi

Trong Bloxorz, người chơi điều khiển khối hộp $1 \times 1 \times 2$, ban đầu đứng thẳng tại ô xuất phát, đến ô đích. Mỗi lệnh lên, xuống, trái hoặc phải làm khối lăn và có thể đổi tư thế; chỉ thắng khi khối còn nguyên vẹn, đứng thẳng trên đích[8].

Ô/cơ chế	Luật chính
Sàn thường; khoảng trống và cầu tắt	Sàn đỡ khối; khoảng trống và cầu tắt thì không. Nếu bất kỳ phần nào của khối không được đỡ, ván chơi thua.
Ô dễ vỡ	Đỡ khối đang nằm hoặc cube nhỏ; khối nguyên vẹn đứng trên ô này làm thua.
Cầu và công tắc	Cầu chỉ đỡ khối khi bật. Công tắc có thể bật, tắt hoặc đảo trạng thái một hay nhiều cầu.
Ô tách và tự ghép	Khối nguyên vẹn đứng thẳng trên ô tách chuyển thành hai cube tại các vị trí quy định; hai cube tự ghép khi kề cạnh theo hàng hoặc cột, không ghép khi chỉ kề chéo.

Bảng 2: Các luật và cơ chế chính của Bloxorz

Công tắc nhẹ được kích hoạt khi phần khối vừa di chuyển đặt lên ô, kể cả cube nhỏ; công tắc nặng

chỉ nhận khối nguyên vẹn đứng thẳng. Tác dụng được giữ sau khi rời công tắc, còn cube đứng yên không kích hoạt lại công tắc nhẹ khi cube kia di chuyển. Cube nhỏ không kích hoạt công tắc nặng hoặc ô tách. Môi trường là hữu hạn, xác định và quan sát đầy đủ.

3.2 Đặc tả JSON và ràng buộc

Đầu vào mô tả một màn bằng năm trường sau; các mảng cơ chế có thể rỗng.

Trường	**Nội dung**
`id`	Định danh màn chơi.
`board`	Các hàng của bàn chơi dưới dạng mảng chuỗi.
`bridges`	Các nhóm cầu, mỗi nhóm có `id`, `cells` và trạng thái ban đầu `active`.
`switches`	Các công tắc tại `at`, với `effects` xác định cầu và chế độ tác động.
`splitters`	Các ô tách tại `at`, với `destinations=[p1,p2]` lần lượt chỉ vị trí của cube 1 và cube 2.

Bảng 3: Các trường chính của JSON đầu vào

Ký hiệu trên bàn: `#` là khoảng trống, `.` là sàn, `S` là xuất phát, `G` là đích, `F` là ô dễ vỡ, `B` là cầu, `O/X` lần lượt là công tắc nhẹ/nặng và `T` là ô tách. Tọa độ có dạng `[r,c]`, đánh số hàng và cột từ 0. Bàn là hình chữ nhật, kích thước tối đa 30×30, có đúng một `S` và một `G`.

Mỗi công tắc có `effects` gồm các trường `bridge` và `mode`; `mode` nhận một trong các giá trị `on`, `off`, `toggle`. Tọa độ phải nằm trong bàn; các định nghĩa phải khớp loại ô, ID cầu phải duy nhất và mọi tham chiếu cầu phải tồn tại. Mỗi ô công tắc và ô tách có đúng một định nghĩa; mỗi ô `B` thuộc đúng một nhóm cầu. Hai điểm đến của ô tách không được trùng nhau hoặc kề cạnh.

Ví dụ output của `sample06`:

```
{"status":"solved","actions":["R","R","1R","R"]}
```

Các token `U`, `D`, `L`, `R` chỉ hướng di chuyển của khối ghép; sau khi tách, token như `1U` đến `1R` hoặc `2U` đến `2R` vừa xác định cube vừa chỉ hướng. Không có hành động riêng để chọn cube.

3.3 Ví dụ minh họa

Bảng 4 tóm tắt ba fixture tiêu biểu và chuỗi hành động tương ứng.

Ví dụ	Hành động	Cơ chế minh họa
`sample03`	`R,R`	Khối nằm đi qua ô dễ vỡ.
`sample05`	`R,R,R,R`	Công tắc nặng bật cầu.
`sample06`	`R,R,1R,R`	Tách khối, di chuyển cube 1 rồi tự ghép.

Bảng 4: Các ví dụ minh họa cơ chế trò chơi

Trong `sample06`, `board=["S..T","####","...G"]`; splitter tại `[0,3]` có `destinations=[[2,0],[2,2]]`. Trace của `sample06`: khối đứng tại $(0,0) \to$ nằm trên $(0,1),(0,2) \to$ tách tại $(2,0),(2,2) \to$ cube 1 đi phải và ghép trên $(2,1),(2,2) \to$ đứng tại đích $(2,3)$. Các ví dụ chỉ minh họa luật chơi, không phải kết quả đánh giá hiệu năng.

3.4 Mô hình trạng thái

Với một bàn chơi cố định, trạng thái tìm kiếm được biểu diễn bởi

$$s = (m, p_1, p_2, \boldsymbol{\beta}), \qquad \boldsymbol{\beta} \in \{0,1\}^q.$$

Ở đây $m = 0$ biểu thị khối nguyên vẹn, còn $m = 1$ biểu thị hai cube đã tách; $p_i = (r_i, c_i)$ là tọa độ trên bàn và β_j ghi trạng thái bật/tắt của nhóm cầu thứ j, với q là số nhóm cầu. Khi khối nguyên vẹn, $p_1 = p_2$ biểu diễn tư thế đứng thẳng, còn hai tọa độ kề cạnh biểu diễn tư thế nằm. Để biểu diễn duy nhất khối nằm, chuẩn hóa $p_1 <_{\text{lex}} p_2$. Khi tách, p_1 và p_2 lần lượt là vị trí cube 1 và cube 2, không hoán đổi ID; trạng thái tách cuối lượt thỏa

$$\|p_1 - p_2\|_1 > 1,$$

vì hai cube kề cạnh sẽ tự ghép.
Với tọa độ xuất phát p_S và cấu hình cầu ban đầu $\boldsymbol{\beta}^{(0)}$, trạng thái đầu là

$$s_0 = (0, p_S, p_S, \boldsymbol{\beta}^{(0)}).$$

Bố cục bàn cùng định nghĩa công tắc và splitter là dữ liệu tĩnh dùng chung. Chi phí đường đi, heuristic, parent và cube đang chọn trên GUI không thuộc trạng thái tìm kiếm.

3.5 Thứ tự hiệu ứng trong một lượt

Một lượt tạo trạng thái mới theo $s' = T(s, a)$, trong đó T là hàm chuyển bộ phận: không xác định nếu action sai chế độ hoặc dẫn đến thua.

Gọi d là vector đơn vị ứng với hướng di chuyển. Nếu khối đứng tại p, sau khi lăn nó chiếm hai ô $p + d$ và $p + 2d$. Với khối đang nằm, di chuyển dọc chiều dài làm khối dựng đứng tại ô ngay ngoài đầu khối theo hướng đi; di chuyển vuông góc tịnh tiến cả hai tọa độ thêm d. Khi tách, nếu action chọn cube $i \in \{1, 2\}$, chỉ cube đó di chuyển:

$$p'_i = p_i + d, \qquad p'_{3-i} = p_{3-i}.$$

Các hiệu ứng của lượt được xử lý theo thứ tự sau:

Bước	**Pha xử lý**
1	Kiểm tra action phù hợp chế độ và ván chưa kết thúc.
2	Thực hiện di chuyển hình học.
3	Kiểm tra chỗ đỡ theo cấu hình cầu cũ.
4	Kích hoạt công tắc và cập nhật trạng thái cầu.
5	Tách và dịch chuyển nếu khối nguyên vẹn đứng thẳng trên `T`.
6	Kiểm tra chỗ đỡ của toàn bộ khối theo vị trí và cấu hình cầu mới.
7	Ghép hai cube nếu chúng kề cạnh.
8	Kiểm tra đích theo định nghĩa ở Mục 3.6.

Bảng 5: Thứ tự xử lý một lượt

Khi tách, chỉ cube vừa di chuyển kích hoạt công tắc; cube đứng yên vẫn phải qua kiểm tra chỗ đỡ. Nếu nhiều công tắc được kích hoạt, xử lý theo thứ tự tọa độ hàng rồi cột, và theo thứ tự khai báo của từng danh sách `effects`. Với cầu thứ j, các chế độ cập nhật là `on` : $\beta_j \leftarrow 1$, `off` : $\beta_j \leftarrow 0$, và `toggle` : $\beta_j \leftarrow 1 - \beta_j$.

Không kích hoạt thêm công tắc hoặc splitter tại điểm đến dịch chuyển, cũng không chạy lại hiệu ứng sau khi ghép. Hành động không tạo successor nếu đi ra ngoài bàn, vào khoảng trống hoặc cầu tắt, làm hai cube chồng nhau, hay khiến khối nguyên vẹn đứng trên ô `F`.

3.6 Hàm chi phí và đích

Với p_G là tọa độ ô đích, điều kiện thắng được định nghĩa bởi

$$\text{Goal}(s) \iff m = 0 \wedge p_1 = p_2 = p_G.$$

Mọi trạng thái hợp lệ thỏa điều kiện này đều là trạng thái đích; không yêu cầu cấu hình cầu cố định. Gọi S_G là tập các trạng thái đích.
Với hàm chuyển T tại Mục 3.5, tập đường đi thắng hợp lệ được định nghĩa là

$$\Pi_G(s_0) = \left\{ \pi = (a_1, \ldots, a_k) \;\middle|\; \begin{array}{l} s_i = T(s_{i-1}, a_i), \quad 1 \le i \le k, \\ s_k \in S_G, \quad s_i \notin S_G, \quad 0 \le i < k \end{array} \right\}.$$

Mỗi đường đi dừng ngay khi đạt đích. Trong mô hình chi phí đơn vị của BFS, DFS và A*,

$$c_0(s, a, s') = 1, \qquad C_0(\pi) = L(\pi) = |\pi| = \sum_{i=1}^{k} c_0(s_{i-1}, a_i, s_i) = k.$$

Gọi π_0^* là một đường đi tối ưu theo số action:

$$\pi_0^* \in \arg\min_{\pi \in \Pi_G(s_0)} C_0(\pi).$$

BFS và A* tối ưu C_0 khi các điều kiện của thuật toán được thỏa mãn, gồm điều kiện heuristic tại Mục 4.8; DFS không bảo đảm tối ưu.

Chi phí UCS Với một chuyển tiếp hợp lệ (s, a, s'), cost UCS được định nghĩa là

$$c_U(s, a, s') = 1 + \chi_B(s, a, s') + 2\chi_F(s, a, s') + 3\chi_T(s, a, s'), \qquad \chi_B, \chi_F, \chi_T \in \{0, 1\}.$$

Gọi $\mathcal{B}$ và $\mathcal{F}$ lần lượt là tập tọa độ ô cầu và ô dễ vỡ trên bản đồ cố định. Tập ô được xét trong bước đi là

$$\mathcal{Q}(s, a, s') = \begin{cases} \{p_1', p_2'\}, & m = 0, \\ \{p_i + d(a)\}, & m = 1 \text{ và } a \text{ chỉ định cube } i, \end{cases}$$

trong đó $d(a)$ là vector hướng của action. Khi khối còn nguyên, hai tọa độ ở trạng thái kế tiếp được xét; nếu bước đó kích hoạt splitter thì chúng là hai điểm đến sau dịch chuyển. Khi khối đã tách, chỉ ô đến của cube được action chỉ định được xét, không tính cube đứng yên; nếu hai cube tự ghép ở cuối bước, vẫn dùng ô đến này trước khi chuẩn hóa tọa độ.

Các chỉ báo được xác định bởi

$$\chi_B(s,a,s') = \mathbf{1}[\mathcal{Q}(s,a,s') \cap \mathcal{B} \neq \emptyset],$$
$$\chi_F(s,a,s') = \mathbf{1}[\mathcal{Q}(s,a,s') \cap \mathcal{F} \neq \emptyset],$$
$$\chi_T(s,a,s') = \mathbf{1}[m = 0 \wedge m' = 1].$$

Vì mỗi chỉ báo chỉ nhận 0 hoặc 1, mỗi loại phụ phí chỉ được cộng một lần trong một bước; các loại khác nhau được cộng dồn. Nằm trên ô tách không tự phát sinh phụ phí tách, và tự ghép không có phụ phí riêng. Chuyển tiếp không hợp lệ bị loại khỏi đồ thị, không được gán cost.

Các trọng số là ưu tiên do nhóm lựa chọn để cân bằng số bước với việc đi qua cầu, ô dễ vỡ và kích hoạt tách; chúng không biểu diễn xác suất thua hay chi phí vật lý. Với $\pi = (a_1, \ldots, a_k)$, tổng cost và mục tiêu UCS là

$$C_U(\pi) = \sum_{i=1}^{k} c_U(s_{i-1}, a_i, s_i), \qquad \pi_U^* \in \arg\min_{\pi \in \Pi_G(s_0)} C_U(\pi).$$

UCS tối ưu C_U, không nhất thiết tối ưu độ dài L. Có thể tính C_U cho lời giải của thuật toán khác để so sánh cùng thước đo mà không đổi mục tiêu tìm kiếm của thuật toán đó. Vì c_U chỉ phụ thuộc s, a, s' và bản đồ cố định, không cần bổ sung lịch sử vào khóa trạng thái.

3.7 So sánh trạng thái lặp

Trong cùng một màn, sau chuẩn hóa tọa độ ở Mục 3.4, hai trạng thái được xem là trùng khi

$$s \sim t \iff (m, p_1, p_2, \boldsymbol{\beta}) = (m', p_1', p_2', \boldsymbol{\beta}').$$

Cần giữ $\boldsymbol{\beta}$ vì cấu hình cầu có thể làm thay đổi các nước đi hợp lệ; khi tách, không hoán đổi ID cube vì token như `1R` chỉ định cube 1. Chi phí tích lũy g, heuristic h và parent được quản lý riêng, không thuộc khóa trạng thái.

BFS và DFS dùng tập đã thăm; UCS và A* giữ chi phí tốt nhất đã biết, cập nhật khi tìm được đường rẻ hơn. Giả mã và quy tắc frontier được trình bày trong phần thuật toán.

4 Thiết kế hệ thống và các thuật toán tìm kiếm

4.1 Kiến trúc tổng thể và luồng dữ liệu

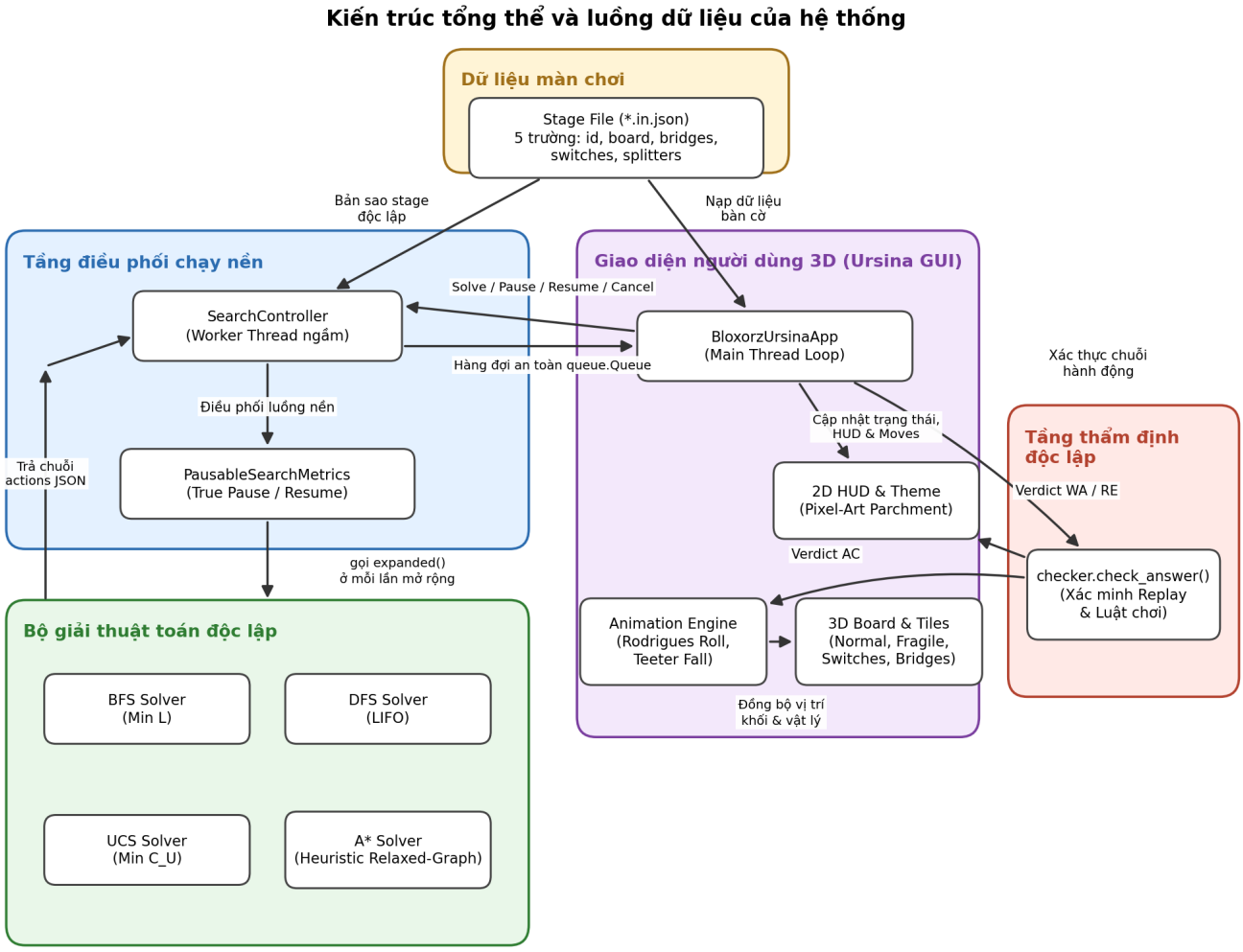
Hệ thống gồm bốn thành phần có trách nhiệm tách biệt:

1. **Game engine** (`game/engine.py`): cài đặt luật chơi cho chế độ tương tác, gồm quản lý tư thế khối, di chuyển hình học, kiểm tra chỗ đỡ, ô dễ vỡ, cầu và công tắc, tách và ghép khối; engine sử dụng luật chuyển trạng thái trong `checker.Replay`.

2. **Bộ giải** (`algorithms/BFS`, `DFS`, `UCS`, `ASTAR`): không import thành phần giao diện nào. Bộ giải nhận dữ liệu màn, tìm kiếm trên không gian trạng thái $s = (m, p_1, p_2, \boldsymbol{\beta})$ ở Mục 3.4 và báo cáo số liệu qua `SearchMetrics`.

3. **Checker** (`checker.py`): đọc danh sách action trong JSON do solver bất kỳ sinh ra, kiểm tra replay bằng các luật trong `checker.Replay` và gán verdict `AC`, `WA` hoặc `RE` trước khi lời giải được đưa vào giao diện.

4. **Giao diện 3D** (`game/ursina_ui/`): xây dựng trên Ursina (dựa trên Panda3D), gồm bàn chơi 3D, bộ hoạt ảnh, HUD và bộ điều khiển tìm kiếm chạy nền.



Hình 1: Kiến trúc tổng thể và luồng dữ liệu giữa dữ liệu màn, bộ giải, checker và giao diện.

Luồng dữ liệu giữa các thành phần như sau.

- **Nạp dữ liệu:** giao diện đọc file `*.in.json` (năm trường `id`, `board`, `bridges`, `switches`, `splitters`; xem Mục 3), dựng bàn chơi 3D và khởi tạo trạng thái đầu $s_0 = (0, p_S, p_S, \boldsymbol{\beta}^{(0)})$. Schema, biểu diễn trạng thái và thứ tự hiệu ứng được trình bày tại Mục 3 và Mục 3.5.

- **Yêu cầu giải:** khi người dùng chọn thuật toán và bấm Giải, giao diện sao chép dữ liệu màn và chuyển cho `SearchController`.

- **Thực thi bất đồng bộ:** `SearchController` tạo một luồng nền (`worker_thread`, daemon). Solver gọi `metrics.expanded()` mỗi lần mở rộng trạng thái; hàm này là điểm kiểm tra phối hợp (Mục 4.3).

- **Giao tiếp giữa các luồng:** kết quả được đóng gói trong `SearchTaskResult` và đưa vào `queue.Queue`. Luồng chính của Ursina gọi `poll()` trong `update()` ở mỗi khung hình nên không bị chặn.
- **Thẩm định và trình chiếu:** lời giải đi qua `checker.check_answer()`. Nếu verdict là `AC`, chuỗi action được chuyển cho `replay_actions` để bộ hoạt ảnh phát lại trên bàn chơi; verdict `WA` hoặc `RE` chỉ được báo trên HUD.

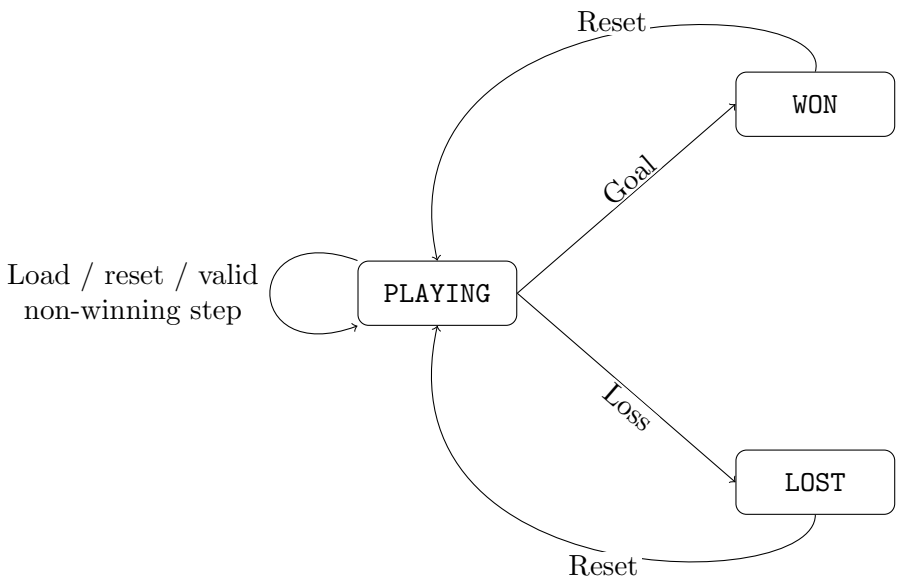


Hình 2: Bảng điều khiển chọn một trong bốn thuật toán BFS, DFS, UCS và A*.

4.2 Chơi thủ công, điều khiển và quản lý trạng thái

4.2.1 Vòng đời ván chơi và tư thế khối

Biến `q` (GameStatus) của engine chỉ nhận `PLAYING`, `WON` hoặc `LOST`. Nạp hoặc reset màn đặt `q=PLAYING`; bước hợp lệ chưa thắng giữ trạng thái này, đạt đích chuyển sang `WON`, còn mất chỗ đỡ, rơi hoặc khối nguyên đứng trên ô dễ vỡ chuyển sang `LOST`. Reset từ mọi trạng thái trở về `PLAYING`; Undo khôi phục trạng thái đã lưu trước đó nếu còn history.



`SPLIT` là tư thế khi $m = 1$, không phải GameStatus. `FALLING` là hoạt ảnh của lớp trình diễn, không phải GameStatus; engine có thể đã ở `LOST` khi hoạt ảnh còn chạy.

Hình 3: Ba trạng thái GameStatus của engine; SPLIT và FALLING chỉ được chú thích ngoài đồ thị.

Tư thế của khối được biểu diễn riêng với GameStatus:

- `UPRIGHT`: khối nguyên vẹn đứng trên một ô, $m = 0$ và $p_1 = p_2$.
- `HORIZONTAL`: hai ô của khối cùng hàng.
- `VERTICAL`: hai ô của khối cùng cột.
- `SPLIT`: hai cube tách rời, $m = 1$.

Khi khối nguyên vẹn đứng thẳng trên ô tách `T`, engine đặt $m = 1$ và hai cube tại các `destinations`. Phím `Space` đổi `selected_cube` (1 hoặc 2). Khi hai cube kề cạnh, $\|p_1 - p_2\|_1 = 1$, engine tự ghép chúng theo luật ở Mục 3.5.



Hình 4: Khối đã tách thành hai cube; cube đang điều khiển có màu đậm hơn.

4.2.2 Hoàn tác (Undo) và đặt lại (Reset)

Ngăn xếp `history` lưu mỗi bước dưới dạng (`State` trước bước, `selected_cube` trước bước, action, GameStatus trước bước, `lost_reason` trước bước). `State` bất biến lưu tọa độ, cờ tách và cờ cầu. Phím `Z` hoặc `U` Undo chỉ khi `history` không rỗng: lấy bản ghi gần nhất, khôi phục các giá trị và giảm `move_count` một bước.

Phím `R` Reset đưa ván chơi về `initial_state`, đặt GameStatus thành `PLAYING`, `selected_cube`=1 và `move_count`=0, xóa `history` và `lost_reason`. GUI đồng bộ lại khối, cầu và gạch dễ vỡ. Phím `C` đưa camera về mặc định.



Hình 5: Giao diện 3D khi chơi thủ công màn 01.

4.3 Tích hợp solver, luồng nền và tạm dừng tìm kiếm

4.3.1 Lý do dùng luồng nền

Một số lần giải tốn từ vài trăm mili-giây đến vài giây; ví dụ BFS mở rộng 5883 trạng thái và UCS mở rộng 5977 trạng thái trên Stage 10 (Phụ lục, bảng expanded nodes). Nếu chạy trên luồng chính, vòng lặp render của Ursina bị chặn và cửa sổ ngừng phản hồi. Vì vậy lớp `SearchController` (`game/search_controller.py`) chạy solver trên một `worker_thread` riêng.

4.3.2 Tạm dừng và tiếp tục ngay trong lúc tìm kiếm

Ngoài tạm dừng replay, giao diện cho phép tạm dừng chính tiến trình tìm kiếm: thuật toán không bị hủy rồi chạy lại từ đầu, và các cấu trúc dữ liệu của nó (frontier hoặc heap, tập đã thăm hoặc `best_cost`, bảng `parent`) được giữ nguyên trong ngăn xếp của luồng.

Để không phải sửa mã của từng thuật toán, nhóm cài `PausableSearchMetrics` kế thừa `SearchMetrics`. Hàm `expanded()` được các solver gọi ở mỗi lần mở rộng nên đóng vai trò điểm kiểm tra phối hợp:

```
def expanded(self) -> None:
    # 1. Cancel request (reset / stage change)
```

```
    if self._cancel_event.is_set():
        raise SearchCancelledError("Search cancelled.")

    # 2. Pause request: block here, search state is kept in memory
    if not self._pause_event.is_set():
        self._is_currently_paused = True
        t0 = perf_counter_ns()
        self._pause_event.wait()
        self._is_currently_paused = False
        self._total_paused_ns += perf_counter_ns() - t0
        if self._cancel_event.is_set():
            raise SearchCancelledError("Cancelled while paused.")

    # 3. Timeout, counting active time only
    if self._timeout_sec is not None:
        active_ns = (perf_counter_ns() - self._start_ns) - self._total_paused_ns
        if active_ns >= self._timeout_sec * 1_000_000_000:
            raise SearchTimeoutError(f"Timeout ({self._timeout_sec:.1f}s)")

    super().expanded()
```

Khi người dùng bấm **Tạm Dừng**, giao diện gọi `search_controller.pause()`, xóa cờ `_pause_event`; luồng solver dừng tại `wait()` ở lần mở rộng kế tiếp. Khi bấm **Tiếp Tục**, `resume()` đặt lại cờ và solver chạy tiếp từ đúng trạng thái đang giữ.

4.3.3 Thời gian tạm dừng và timeout

Tổng thời gian tạm dừng được cộng dồn vào t_{pause} (`_total_paused_ns`) và bị loại khỏi thời gian báo cáo:

$$\texttt{search_time_ms} = \frac{(t_{\text{now}} - t_{\text{start}}) - t_{\text{pause}}}{10^6}.$$

Kiểm tra timeout cũng dùng thời gian hoạt động $t_{\text{active}} = (t_{\text{now}} - t_{\text{start}}) - t_{\text{pause}}$, nên thời gian người dùng để solver tạm dừng không gây timeout. Cơ chế này chỉ có trong giao diện: benchmark ở Mục 5.2 chạy bằng dòng lệnh với `SearchMetrics` nguyên bản, không đi qua `PausableSearchMetrics`, nên các số đo thời gian và bộ nhớ không chịu overhead đồng bộ luồng của giao diện.

4.3.4 Loại bỏ kết quả cũ khi Reset hoặc đổi màn

Nếu người dùng Reset hoặc đổi màn trong lúc solver đang chạy hoặc đang tạm dừng, hệ thống xử lý như sau.

1. Mỗi lần Reset hoặc đổi màn tăng `_solver_generation` và gọi `search_controller.cancel()`.
2. `cancel()` đặt `_pause_event` để đánh thức luồng đang tạm dừng, rồi đặt `_cancel_event`; solver thoát qua `SearchCancelledError` ở lần gọi `expanded()` kế tiếp.
3. Khi một kết quả đến hàng đợi `_results_queue`, hàm `_poll_astar_results()` đối chiếu thế hệ và mã màn, và bỏ kết quả không còn hiệu lực:

```
if res.generation != self._solver_generation or self.engine.stage_id != self.
    _solver_stage_id:
     return  # stale result, do not load into replay
```

Nhờ đó lời giải của màn cũ không được phát lên màn mới.

4.4 Hoạt ảnh 3D và đồng bộ replay

4.4.1 Chuyển động lăn khối

Khối $1 \times 1 \times 2$ lăn bằng cách quay quanh một cạnh đáy tiếp xúc với sàn. Cạnh này thay đổi theo tư thế engine (`UPRIGHT`, `HORIZONTAL`, `VERTICAL`) và hướng lăn (`U`, `D`, `L`, `R`). Để khối không xuyên sàn hoặc bị nhấc lên, tâm khối được xoay quanh trục cạnh đáy theo công thức Rodrigues với trục đơn vị $\mathbf{k}$ và góc θ:

$$\mathbf{v}_{\text{rot}} = \mathbf{v}\cos\theta + (\mathbf{k} \times \mathbf{v})\sin\theta + \mathbf{k}(\mathbf{k} \cdot \mathbf{v})(1 - \cos\theta).$$

Tiến trình $t \in [0, 1]$ của chuyển động đi qua hàm làm mượt `ease_out_cubic`

$$f(t) = 1 - (1 - t)^3,$$

kèm hiệu ứng co giãn nhẹ (squash and stretch) khi khối chạm sàn.

4.4.2 Rơi khỏi mép sàn

Khi khối đi quá mép, hoạt ảnh rơi gồm hai giai đoạn.

1. **Chao đảo ở mép:** khối xoay quanh gờ cuối cùng của ô còn đỡ với tốc độ góc $\omega = 280°/\text{s}$ và trôi nhẹ ra ngoài với vận tốc 2.4 đơn vị/s để không cắt vào ô gạch.

2. **Rơi tự do:** khi góc nghiêng vượt 45°, khối rời điểm tựa và rơi với gia tốc $g = -28$ đơn vị/s^2 cùng chuyển động xoay nhiều trục cho đến khi ra khỏi tầm nhìn camera.



Hình 6: Khối đi quá mép và bị rơi; có hoạt ảnh rơi và màn hình thử lại.

4.4.3 Hiệu ứng đặc biệt

- **Ô dễ vỡ:** khi khối đứng thẳng trên ô `F`, ô vỡ thành các mảnh nhỏ rơi xuống và khối rơi theo phương thẳng đứng.

- **Chiến thắng:** khi khối đứng thẳng lọt vào ô `G`, khối chìm dần vào hố đích và hệ thống hạt (`ParticlePool`) tạo hiệu ứng confetti.



Hình 7: Hiệu ứng chiến thắng khi khối đứng thẳng lọt vào ô đích.

4.4.4 Replay lời giải

Bộ điều khiển replay của GUI dùng lại pipeline hoạt ảnh của chế độ chơi thủ công. Thanh replay có nút phát/tạm dừng (hoặc phím `Space`), nút tiến một bước, nút lùi một bước, và hiển thị bước hiện tại trên tổng số bước, ví dụ `Bước 12 / 57`. Vì mọi lời giải đều qua checker trước khi replay, trạng thái của engine và hình ảnh 3D được kiểm tra khớp nhau; trace mẫu và ma trận 19 ca kiểm thử nằm ở Phụ lục C. Hướng dẫn cài đặt, các lệnh khởi chạy và bảng phím nằm ở Mục A.3.

4.5 Breadth-First Search (BFS)

Ý tưởng và giả mã Breadth-First Search (BFS), hay tìm kiếm theo chiều rộng, mở rộng trạng thái theo khoảng cách tăng dần từ trạng thái đầu. Hàng đợi FIFO giữ thứ tự này[9].

Algorithm 1 BFS trong chương trình

```
Q ← [s_0]
Seen ← {s_0}
Parent[s_0] ← NIL
while Q không rỗng do
    s ← PopFront(Q)
    if Goal(s) then
        return ReconstructPath(Parent, s)
    end if
    for all (a, s') trong Successors(s) do
        if s' ∉ Seen then
            Seen ← Seen ∪ {s'}
            Parent[s'] ← (s, a)
            PushBack(Q, s')
        end if
    end for
end while
return failure
```

BFS đánh dấu trạng thái trong `Seen` (visited) ngay khi đưa vào hàng đợi và lưu trạng thái cha cùng action trong `Parent`; các thông tin này ngăn lặp và dùng để dựng lại lời giải.

Tính đầy đủ, tối ưu và độ phức tạp Trên đồ thị hữu hạn với hữu hạn successor mỗi trạng thái, mỗi trạng thái chỉ được đưa vào hàng đợi một lần nên BFS đầy đủ. FIFO xử lý các lớp độ sâu theo thứ tự; khi mọi cạnh có chi phí đơn vị, đích đầu tiên lấy khỏi hàng đợi cho lời giải có ít action nhất. Gọi $n = |V|$ là số trạng thái đạt được và $e = |E|$ là số cạnh hợp lệ; thời gian là $O(n + e)$, bộ nhớ là $O(n)$ do lưu frontier, visited và parent toàn cục. Thứ tự successor chỉ quyết định lựa chọn giữa các lời giải cùng độ dài.

4.6 Depth-First Search (DFS)

Ý tưởng và giả mã Depth-First Search (DFS), hay tìm kiếm theo chiều sâu, ưu tiên mở rộng nhánh mới nhất trước khi quay lại các nhánh khác[9]. Bản cài đặt dùng stack lặp LIFO, không phải DFS đệ quy hay tìm kiếm sâu dần (IDS).

Algorithm 2 DFS trong chương trình

```
Stack ← [s_0]
Seen ← {s_0}
Parent[s_0] ← NIL
while Stack không rỗng do
    s ← Pop(Stack)
    if Goal(s) then
        return ReconstructPath(Parent, s)
    end if
    for all (a, s') trong Successors(s) do
        if s' ∉ Seen then
            Seen ← Seen ∪ {s'}
            Parent[s'] ← (s, a)
            Push(Stack, s')
        end if
    end for
end while
return failure
```

DFS đánh dấu trạng thái trong `Seen` (visited) ngay khi đẩy vào stack và lưu cha cùng action trong `Parent` để dựng lại đường đi.

Tính đầy đủ, tối ưu và độ phức tạp Trên đồ thị hữu hạn với hữu hạn successor mỗi trạng thái, visited toàn cục bảo đảm DFS đầy đủ và không lặp chu trình. DFS không bảo đảm tối ưu số action vì thứ tự duyệt không dựa trên độ dài đường đi. Mỗi trạng thái được đẩy một lần nên mỗi cạnh đi ra được xét tối đa một lần; với $n = |V|$ trạng thái và $e = |E|$ cạnh, thời gian là $O(n + e)$. Stack, visited và parent toàn cục cần $O(n)$ bộ nhớ, không chỉ phụ thuộc độ sâu. Thứ tự successor ảnh hưởng nhánh được xét trước và có thể làm thay đổi lời giải.

4.7 Uniform-Cost Search (UCS)

Ý tưởng và giả mã UCS mở rộng trạng thái có chi phí tích lũy g_U nhỏ nhất trong min-heap[9]. Với chuyển tiếp hợp lệ (s, a, s'), chi phí được cập nhật theo

$$g_U(s') = g_U(s) + c_U(s, a, s'),$$

trong đó c_U được định nghĩa tại Mục 3.6. Khóa trạng thái vẫn là $(m, p_1, p_2, \boldsymbol{\beta})$ tại Mục 3.4; g_U, lịch sử và cube đang chọn trên GUI không thuộc khóa. Khi khối tách, ID cube đã nằm trong action nên

không có cạnh chọn cube riêng.

Algorithm 3 UCS với chi phí tốt nhất đã biết

```
Best[·] ← +∞
Best[s_0] ← 0; Parent[s_0] ← NIL
PQ ← {(s_0, 0)}                                   ▷ min-heap theo chi phí
while PQ không rỗng do
    (s, g_q) ← PopMin(PQ)
    if g_q > Best[s] then
        continue
    end if
    if Goal(s) then
        return (ReconstructPath(Parent, s), g_q)
    end if
    for all (a, s') trong Successors(s) do
        candidate ← g_q + c_U(s, a, s')
        if candidate < Best[s'] then
            Best[s'] ← candidate; Parent[s'] ← (s, a)
            Push(PQ, (s', candidate))
        end if
    end for
end while
return failure
```

Khi hai bản ghi có cùng cost, bản cài ưu tiên bản ghi được thêm trước; số action không phải tiêu chí phụ. Trên đồ thị trạng thái hữu hạn với mọi cạnh $c_U \geq 1$, cập nhật chi phí tốt nhất và bỏ bản ghi lỗi thời bảo đảm trạng thái được lấy ra bằng bản ghi còn hiệu lực có chi phí tối ưu. Vì vậy, đích đầu tiên được lấy khỏi heap tối thiểu hóa C_U. UCS đầy đủ nếu có lời giải và không bị giới hạn tài nguyên.

Với n trạng thái, e cạnh và giả thiết thao tác trạng thái, sinh successor, tính cost có chi phí cố định, thời gian là $O((n+e)\log(n+1))$; bộ nhớ là $O(n+e)$, gồm heap và các bảng lưu trữ. Bậc ra tối đa là 8, nên $e = O(n)$ và bộ nhớ tương đương $O(n)$. Solver xuất tổng cost trong `metrics.weighted_cost`; số action và trường cost cũ của runner vẫn biểu diễn số bước, cần phân biệt khi báo cáo.

4.8 A*

1. Ý tưởng và giả mã Dựa trên mô hình trạng thái, hàm chuyển và điều kiện đích tại Mục 3.4, Mục 3.5 và Mục 3.6, A* tìm đường thắng có ít action nhất với chi phí mỗi action bằng 1[10]. Thuật

toán ưu tiên trạng thái theo

$$f(s) = g(s) + h(s),$$

trong đó $g(s)$ là chi phí tốt nhất đã phát hiện từ trạng thái đầu, còn $h(s)$ ước lượng chi phí còn lại. Khi tìm được đường rẻ hơn tới cùng trạng thái, A* cập nhật g và trạng thái cha. Mỗi mục trong hàng đợi ưu tiên $OPEN$ lưu trạng thái cùng giá trị g tại thời điểm được thêm; mục được lấy theo f, và trường hợp hòa được xét theo thứ tự thêm.

Algorithm 4 A* với heuristic khoảng cách thư giãn

Dựng G_r và D; đặt $h(s) \leftarrow D(P(s))$, mặc định $+\infty$
if $h(s_0) = +\infty$ **then**
    return failure
end if
$best$ mặc định $+\infty$; $best[s_0] \leftarrow 0$; $parent[s_0] \leftarrow$ NIL; $OPEN \leftarrow \emptyset$
Push$(OPEN, (s_0, 0))$ với ưu tiên $h(s_0)$
while $OPEN$ không rỗng **do**
    $(s, g_q) \leftarrow$ PopMin$(OPEN)$
    if $g_q \neq best[s]$ **then**
        continue
    end if
    if $Goal(s)$ **then**
        return truy vết từ $parent$
    end if
    for all (a, s') là successor hợp lệ của s từ model chung **do**
        $g' \leftarrow g_q + 1$
        if $g' < best[s']$ và $h(s') < +\infty$ **then**
            $best[s'] \leftarrow g'$; $parent[s'] \leftarrow (s, a)$
            Push$(OPEN, (s', g'))$ với ưu tiên $g' + h(s')$
        end if
    end for
end while
return failure

2. Thiết kế heuristic Heuristic là khoảng cách chính xác trong một bài toán thư giãn, dùng làm cận dưới cho số action còn lại trong trò chơi thật. Mô hình thư giãn đổi các ô `F`, `B`, `O`, `X` thành sàn thường, đồng thời bỏ trạng thái cầu và tác dụng công tắc; vẫn giữ khoảng trống, biên bàn, đích, hình học di chuyển, chế độ tách/ghép, ID cube, splitter và các điểm đến dịch chuyển. Phép chiếu trạng thái là

$$P(m, p_1, p_2, \boldsymbol{\beta}) = (m, p_1, p_2, ()).$$

Gọi $G_r = (V_r, E_r)$ là đồ thị có hướng gồm các trạng thái thư giãn đạt được từ $P(s_0)$, với chi phí mỗi cạnh bằng 1, và $S_{G,r}$ là tập trạng thái đích trong đồ thị đó. Tiền xử lý duyệt tiến để dựng đồ thị và lưu các cạnh tiền nhiệm, sau đó chạy BFS ngược đa nguồn từ $S_{G,r}$ để tính

$$h(s) = D(P(s)) = \mathrm{dist}_{G_r}\big(P(s), S_{G,r}\big),$$

trong đó $h(s) = +\infty$ nếu không có đường tới đích thư giãn. Cạnh tiền nhiệm cần thiết vì chuyển tiếp qua splitter có thể không đảo được bằng một action ngược. Bảng khoảng cách được dựng một lần cho mỗi lần giải và tra cứu trong suốt A*, không chạy lại BFS khi tính $h(s)$.

3. Tính đúng đắn và tính tối ưu Chỉ xét các trạng thái thật đạt được từ s_0 trong màn đang giải; gọi $d^*(s)$ là số action tối thiểu còn lại tới đích, hoặc $+\infty$ nếu không có đường. Mỗi chuyển tiếp thật hợp lệ $s \xrightarrow{a} s'$ được bảo toàn sau phép chiếu thành chuyển tiếp thư giãn $P(s) \xrightarrow{a} P(s')$ cùng chi phí 1: phép thư giãn chỉ bỏ các hạn chế do ô dễ vỡ, cầu và công tắc, còn giữ biên, khoảng trống, chuyển động, ID cube, splitter và tự ghép. Đích thật cũng chiếu thành đích thư giãn; do đó ảnh chiếu của mọi trạng thái thật đạt được nằm trong V_r.

Với mọi đường thật tới đích dài k, phép chiếu tạo một đường thư giãn cùng độ dài. Vì h là khoảng cách ngắn nhất trong đồ thị thư giãn, $h(s) \le k$ và do đó $h(s) \le d^*(s)$; tại trạng thái đích, $h(s) = 0$. Như vậy heuristic admissible. Trên mỗi cạnh thật hợp lệ, cạnh chiếu tương ứng cho $h(s) \le 1 + h(s')$, nên heuristic cũng consistent[10]; các trường hợp không tới đích được hiểu theo số thực mở rộng với giá trị $+\infty$.

Nếu $h(s) = +\infty$, trạng thái s không thể tới đích thật và được loại an toàn. Trên đồ thị hữu hạn, với chi phí đơn vị, heuristic này cùng quy tắc cập nhật *best*, loại mục heap lỗi thời và chỉ kiểm tra đích khi lấy mục còn hiệu lực bảo đảm A* đầy đủ và tối ưu số action, miễn là không bị giới hạn tài nguyên.

4. Độ phức tạp Đặt N là kích thước đầu vào; $n_r = |V_r|$, $e_r = |E_r|$ là số đỉnh và cạnh của đồ thị thư giãn; $n = |V|$, $e = |E|$ là kích thước đồ thị thật đạt được; và L là độ dài đường đi trả về. Giả sử kích thước trạng thái và chi phí sinh/kiểm tra chuyển tiếp là hằng số, tra bảng băm có chi phí kỳ vọng hằng số, và $OPEN$ dùng heap nhị phân. Các chặn tiệm cận được tóm tắt ở Bảng 6.

Bảng 6: Độ phức tạp của A*

Đại lượng	Chặn tiệm cận
Tiền xử lý	$T_{\text{pre}} = O(N + n_r + e_r)$
Toàn thuật toán	$T_{\text{total}} = O\big(N + n_r + e_r + (n+e)\log(n+1) + L\big)$
Bộ nhớ	$M_{\text{total}} = O(N + n_r + e_r + n + L)$

Tiền xử lý gồm dựng đồ thị thư giãn và BFS ngược. Tính nhất quán cho phép mỗi trạng thái thật được mở rộng tối đa một lần qua mục còn hiệu lực; các mục heap cũ vẫn được tính trong chặn thời gian. Vì mỗi trạng thái có tối đa tám successor, $e = O(n)$. Chặn bộ nhớ tính cả heap, bảng khoảng cách, *best*, *parent* và đường đi; M_{total} là chặn trên cộng an toàn, không phải số đo bộ nhớ thực tế. Heuristic có thể giảm số trạng thái mở rộng, nhưng tiền xử lý vẫn tiêu tốn thời gian và bộ nhớ nên cần đánh giá tổng chi phí ở phần thực nghiệm.

4.9 So sánh các thuật toán

Các thuật toán dùng chung mô hình trạng thái, hàm chuyển và điều kiện đích tại Mục 3.4, 3.5 và 3.6; chi tiết lần lượt được trình bày tại Mục 4.5–4.8.

Bảng 7: So sánh lý thuyết các thuật toán tìm kiếm

Thuật toán	Thứ tự xét	Đầy đủ	Mục tiêu tối ưu
BFS	FIFO; theo độ sâu tăng dần.	Có trên đồ thị hữu hạn với visited.	Tối thiểu số action L, tương đương chi phí đơn vị C_0.
DFS	LIFO; ưu tiên đi sâu.	Có trên đồ thị hữu hạn với visited toàn cục.	Không bảo đảm tối thiểu L hoặc C_U.
UCS	Ưu tiên trạng thái có g_U nhỏ nhất.	Có trên đồ thị hữu hạn vì $c_U \geq 1$.	Tối thiểu tổng cost C_U; không bảo đảm ít action nhất.
A*	Ưu tiên trạng thái có $f = g_0 + h$ nhỏ nhất.	Có trên đồ thị hữu hạn khi thỏa các điều kiện tại Mục 4.8.	Tối thiểu L theo chi phí đơn vị khi thỏa các điều kiện tại Mục 4.8; bản hiện tại không tối ưu C_U vì g_0 cộng 1 mỗi action.

Ghi chú: Các bảo đảm giả sử thuật toán được thực hiện đúng và không bị dừng do giới hạn tài nguyên.

Trong các chặn dưới đây, n, e lần lượt là số trạng thái và cạnh thật đạt được; n_r, e_r là số trạng thái và cạnh của đồ thị thư giãn; N là kích thước đầu vào; L là độ dài lời giải.

Bảng 8: Độ phức tạp lý thuyết của các thuật toán

Thuật toán	Thời gian	Bộ nhớ
BFS	$O(n+e)$	$O(n)$
DFS	$O(n+e)$	$O(n)$
UCS	$O((n+e)\log(n+1))$	$O(n+e)$
A*	$O(N+n_r+e_r$ $+(n+e)\log(n+1)+L)$	$O(N+n_r+e_r$ $+n+L)$

Giả sử kích thước trạng thái, sinh cạnh và tính cost có chi phí cố định, tra bảng băm có chi phí kỳ vọng hằng số và dùng heap nhị phân. Các chặn A* gồm tiền xử lý heuristic và truy vết tại Mục 4.8; vì bậc ra tối đa là 8 nên $e \le 8n$ và bộ nhớ UCS $O(n+e)$ tương đương $O(n)$.

Các thuật toán dùng chung luật và điều kiện đích nhưng không cùng mục tiêu tối ưu: BFS và A* tìm lời giải ít bước, còn UCS tính thêm phụ phí cầu, ô dễ vỡ và kích hoạt tách nên có thể chọn đường dài hơn nhưng có C_U thấp hơn. Có thể tính cả L và C_U cho mọi lời giải để so sánh mà không đổi mục tiêu tìm kiếm của thuật toán. Với cùng hàm cost, A* khi $h = 0$ có thứ tự ưu tiên tương ứng UCS; bản A* hiện tại dùng g_0, còn UCS dùng g_U, nên hai bản cài không đồng nhất. Thứ tự duyệt và bảo đảm tối ưu không đủ để kết luận thuật toán nào nhanh hơn hoặc dùng ít bộ nhớ hơn; việc đó cần đánh giá thực nghiệm.

5 Phương pháp thực nghiệm

5.1 Nguồn dữ liệu và độ bao phủ tính năng

Nguồn và vai trò dữ liệu Bộ dữ liệu gồm 6 sample do nhóm xây dựng để minh họa, kiểm tra cơ chế và 33 stage original chuyển đổi từ kho `tkoz0/bloxorz-solver`[7], tạo thành tập đánh giá cố định. Dữ liệu được chuyển sang JSON của dự án, giữ nguyên bố cục và ý nghĩa các cơ chế. Solver chỉ nhận dữ liệu màn, không đọc lời giải tham chiếu; unit tests bổ sung nằm ngoài 39 màn. Trò chơi Bloxorz được ghi công cho Damien Clarke[2].

Phân nhóm và cơ chế Bảng 9 chia 33 stage original thành các nhóm không giao nhau theo thứ tự ưu tiên: splitter, cầu, ô dễ vỡ, rồi nhóm cơ bản.

Bảng 9: Phân nhóm stage original theo cơ chế ưu tiên

Nhóm	Số màn	Đặc điểm
Có splitter	10	Tách, dịch chuyển và ghép; có thể kèm cầu hoặc công tắc.
Không splitter, có cầu	18	Có thể đồng thời chứa ô dễ vỡ.
Không splitter/cầu, có ô dễ vỡ	2	Có ô dễ vỡ nhưng không có splitter hoặc cầu.
Cơ bản	3	Không có splitter, cầu hoặc ô dễ vỡ.

Tổng số là 33; đây là phân nhóm theo cơ chế hiện diện, không phải thang độ khó.

- `sample01`–`sample02`: lăn khối, đổi tư thế và đạt đích.
- `sample03`: khối nằm đi qua ô dễ vỡ.
- `sample04`–`sample05`: công tắc nhẹ/nặng và cập nhật cầu.
- `sample06`: tách, dịch chuyển, điều khiển từng cube và tự ghép.
- Các stage original bổ sung những màn kết hợp nhiều cơ chế.

Unit tests kiểm tra đầu vào và action không hợp lệ, điều kiện thắng/thua, công tắc làm tắt cầu đang đỡ cube đứng yên và nhiều công tắc cùng tác động một cầu. Kiểm thử UCS cũng bao gồm phụ phí cầu, ô dễ vỡ, kích hoạt tách và trường hợp đường dài hơn có tổng cost thấp hơn. Sự hiện diện của một cơ chế không đồng nghĩa mọi hành vi của cơ chế hoặc tương tác GUI đã được kiểm thử.

Lời giải tham chiếu và phạm vi đánh giá Lời giải tham chiếu của cả 39 màn đã replay thành công theo luật dự án; đây là xác nhận hợp lệ, không tự chứng minh tối ưu. Riêng `stage15`, bố cục không đổi nhưng lời giải nguồn được thay bằng đường BFS dài 57 action để phù hợp luật kiểm tra chỗ đỡ sau khi cầu đổi trạng thái. Độ dài tham chiếu phục vụ đối chiếu mục tiêu ít bước của BFS/A*, không phải mốc tối ưu C_U; lời giải UCS dài hơn không vì thế bị xem là sai.

BFS, DFS, UCS và A* được đánh giá trên cùng 33 stage original theo thiết lập tại Mục 5.2, dùng bộ thực nghiệm mới với cost UCS tại Mục 3.6, không trộn kết quả UCS cost đơn vị cũ. Sáu sample là tập kiểm tra bổ sung, không thuộc benchmark chính. Phạm vi phép đo được trình bày tại Mục 5.3.

5.2 Thiết lập thí nghiệm

Bộ benchmark mới ngày 28/10/2026 đánh giá BFS, DFS, UCS với cost tại Mục 3.6 và A* trên cùng 33 stage original. Mỗi cặp thuật toán–stage chạy 5 lần, tổng cộng $4 \times 33 \times 5 = 660$ lượt chính; kết quả dùng đồng bộ từ bộ benchmark mới, không ghép với kết quả cũ của BFS/DFS/A* hoặc UCS cost đơn vị. Sáu sample được kiểm tra riêng bằng cả bốn thuật toán; mỗi thuật toán warmup một lần trên `sample01`. Kiểm thử sample và warmup không thuộc 660 lượt. Solver chỉ nhận dữ liệu màn, không đọc lời giải tham chiếu.

Môi trường thực thi Bảng 10 tóm tắt cấu hình dùng trong thí nghiệm.

Bảng 10: Môi trường phần cứng và phần mềm của thí nghiệm

Thành phần	**Cấu hình**
CPU	AMD Ryzen 7 8845H
CPU logic	16
RAM hệ thống	Xấp xỉ 27,21 GiB
Hệ điều hành	Linux x86_64, kernel 7.0.0-38
Python	CPython 3.12.3
Công cụ đo	`perf_counter_ns`, `tracemalloc`[6]
Cách chạy	Tuần tự, mỗi lượt trong một tiến trình Python mới

GUI không tham gia phép đo; cả bốn thuật toán dùng cùng cơ chế đo thời gian và bộ nhớ.

Quy trình chạy

1. Kiểm tra dữ liệu và chạy kiểm thử; kiểm tra riêng sáu sample với cả bốn thuật toán, rồi warmup mỗi thuật toán một lần trên `sample01`. Lưu riêng các lượt này.
2. Chạy 5 vòng, mỗi vòng xáo trộn các cặp thuật toán–stage bằng seed `20261028`.
3. Với mỗi lượt, khởi chạy solver trong tiến trình mới, truyền JSON đầu vào và thu lời giải cùng metrics. Timeout là 60 giây mỗi tiến trình; không cấu hình giới hạn bộ nhớ riêng.

4. Sau khi solver kết thúc, checker replay lời giải; tính lại cost C_U của mọi lời giải theo Mục 3.6. BFS/A* dùng `require_optimal` để kiểm tra thắng hợp lệ và đối chiếu số action với mốc tham chiếu. DFS chỉ yêu cầu lời giải thắng hợp lệ. UCS không dùng `require_optimal` vì cờ này chỉ kiểm tra số action; checker đối chiếu cost tính lại với `metrics.weighted_cost`. Replay và đối chiếu cost xác nhận cost của đường đi, không tự chứng minh tối ưu toàn cục.

5. Ngoài benchmark, đối chiếu UCS với Dijkstra được viết riêng trên mô hình checker cho 33 stage; bước này không hướng dẫn solver và không tính vào thời gian hoặc bộ nhớ tìm kiếm.

6. Lưu từng lượt và tổng hợp riêng từng cặp thuật toán–stage bằng median/IQR. Với A*, heuristic được dựng lại mỗi lần giải nên thời gian và bộ nhớ gồm tiền xử lý; tính lại cost sau khi solver chạy nằm ngoài cửa sổ đo.

Định nghĩa đại lượng, ranh giới đo, thống kê và cách xử lý dữ liệu thiếu được trình bày tại Mục 5.3; số lượt đạt và kết quả đối chiếu được báo cáo tại Mục 6.5.

5.3 Quy trình và phép đo

Các đại lượng áp dụng cho BFS, DFS, UCS và A* theo Mục 5.2. Thống kê được tính riêng theo từng cặp thuật toán–stage, tối đa năm lượt đủ điều kiện.

Bảng 11: Định nghĩa và phạm vi các đại lượng đo

Đại lượng	**Phạm vi và định nghĩa**	**Đơn vị**
Thời gian giải	Đo quanh lời gọi giải; gồm dựng model, khởi tạo, tiền xử lý heuristic nếu có, tìm kiếm và truy vết. Không gồm parse JSON, tạo/in output, checker hoặc GUI; với UCS, tính cost cạnh trong tìm kiếm được tính vào.	ms
Peak memory	Đỉnh cấp phát Python do `tracemalloc` theo dõi trong lần giải; A* gồm cả tiền xử lý heuristic.	byte (hiển thị MiB)
Expanded nodes	Số lần mở rộng trạng thái không phải đích để sinh successor; không tính bản ghi heap lỗi thời, và với A* không tính việc dựng heuristic.	trạng thái
Độ dài lời giải L	Số action trong đường thắng hợp lệ được checker chấp nhận.	action
Weighted cost C_U	Tổng cost có phụ phí theo Mục 3.6, tính lại khi replay lời giải của cả bốn thuật toán. Với UCS, đối chiếu với `metrics.weighted_cost`.	cost

L và C_U là hai chỉ tiêu khác nhau: UCS tối ưu C_U, BFS/A* tối ưu L, còn DFS không bảo đảm tối ưu cả hai. Cost cũ của checker vẫn biểu diễn số action, không phải weighted cost. Tính lại cost mọi lời giải và đối chiếu Dijkstra riêng nằm ngoài cửa sổ đo solver; bước Dijkstra được mô tả tại Mục 5.2.

Ranh giới đo, median và IQR Cả bốn thuật toán bật `tracemalloc`, nên thời gian có overhead instrumentation. Đây không phải RSS hay tổng RAM tiến trình; các peak theo pha không được cộng. Thời gian và bộ nhớ A* gồm tiền xử lý, còn expanded nodes chỉ tính pha tìm kiếm chính. Elapsed toàn tiến trình gồm cả khởi động và trao đổi JSON nên khác thời gian giải, không thay thế cho nhau.

Với năm giá trị hợp lệ đã sắp $x_{(1)} \le x_{(2)} \le x_{(3)} \le x_{(4)} \le x_{(5)}$,

$$\widetilde{x} = x_{(3)}, \qquad Q_1 = x_{(2)}, \qquad Q_3 = x_{(4)}, \qquad \mathrm{IQR} = Q_3 - Q_1.$$

Quartile dùng quy ước nội suy inclusive[6]. Khi thiếu lượt, ghi rõ $n/5$; median tính trên các lượt hợp lệ, không có dữ liệu thì ghi NA. Với một quan sát, chỉ báo median và ghi IQR là NA. IQR mô tả độ phân tán, không phải khoảng tin cậy; không chọn lượt nhanh nhất hoặc tự loại outlier. Không gộp lượt nhiều stage như các lần lặp cùng bài toán. Khi tổng hợp qua stage, ghi rõ "median của các median theo stage" và dùng cùng tập stage để so sánh. Không cộng median các thành phần để suy ra median tổng A*.

Verdict và dữ liệu thiếu Ba tiêu chí được xét độc lập: (1) verdict xác nhận đường đi — AC, WA, RE hoặc TLE; (2) độ đầy đủ metrics cơ sở; (3) xác minh weighted cost. Các bảng tài nguyên và độ dài dùng lượt AC có metrics cơ sở hợp lệ. Bảng C_U yêu cầu replay tính được cost; riêng UCS còn phải khớp với `metrics.weighted_cost`. Nếu cost UCS sai hoặc thiếu, ghi lỗi xác minh và loại lượt khỏi thống kê C_U; không đổi verdict AC của đường đi thành WA và không gọi lượt đạt đủ tiêu chí. UCS dài hơn tham chiếu không phải WA; AC cùng cost replay đúng cũng không tự chứng minh tối ưu toàn cục.

WA, RE và TLE được lưu riêng, không trộn vào median lượt hoàn thành hợp lệ. Không thay dữ liệu thiếu bằng 0, không gán thời gian tìm kiếm TLE bằng timeout 60 giây và không chạy bổ sung âm thầm để thay lượt lỗi. Kết quả và số lượt đạt được trình bày tại Mục 6.5.

6 Kết quả và thảo luận

Mục 6.1 trình bày thời gian tìm kiếm; bộ nhớ, số trạng thái mở rộng, độ dài lời giải và verdict được xét riêng. Mô hình chi phí đơn vị C_0 khác với cost có phụ phí C_U của UCS tại Mục 3.6.

6.1 Thời gian tìm kiếm

BFS, DFS, UCS và A* chạy trên cùng 33 stage original, năm lượt mỗi cặp; cả 660 lượt đều AC và đủ metrics cơ sở. Thời gian `search_time_ms` được báo cáo bằng median/IQR theo stage, đơn vị ms. A* gồm tiền xử lý heuristic; UCS gồm chi phí tính cost cạnh trong tìm kiếm. Ranh giới đo và thống kê xem tại Mục 5.3; replay, tính lại C_U và đối chiếu tối ưu riêng nằm ngoài thời gian này.

Median của 33 median thời gian theo stage là 35.789 ms với BFS, 18.845 ms với DFS, 40.015 ms với UCS và 47.182 ms với A* tổng. Đây là median của các median theo stage, không phải trung bình hay median của toàn bộ lượt gộp.

Bảng 12 minh họa bốn stage; mỗi ô ghi median (IQR), ms. Dữ liệu đủ 33 stage được trình bày tại phụ lục.

Bảng 12: Thời gian tìm kiếm minh họa theo stage: median (IQR), ms

Stage	BFS	DFS	UCS	A* tổng
08	49.447 (4.227)	9.877 (0.260)	55.632 (0.688)	151.470 (3.451)
15	1417.523 (1198.450)	50.043 (13.366)	582.693 (19.799)	692.192 (2.641)
23	400.831 (589.835)	585.136 (233.222)	461.285 (26.663)	3306.104 (2053.381)
26	1673.590 (584.975)	323.695 (7.677)	1016.199 (31.980)	1773.977 (2347.786)

Theo median chưa làm tròn, DFS nhanh hơn BFS ở 28/33 stage, UCS ở 8/33 và A* tổng ở 7/33. Đây là mô tả của batch, không phải kiểm định ý nghĩa thống kê.

Stage08 minh họa chi phí tiền xử lý: median A* tổng là 151.470 ms, gồm median tiền xử lý 147.999 ms và tìm kiếm chính 2.533 ms. Các median thành phần được tính độc lập nên không nhất thiết cộng lại bằng median tổng.

IQR lớn tại một số stage, đáng chú ý là BFS ở stage15 và A* ở stage23/26, cho thấy thời gian dao động đáng kể trong batch; không thể từ đó kết luận thứ hạng có ý nghĩa thống kê hoặc ổn định ở môi trường khác. Thời gian cần được đọc cùng chất lượng lời giải: BFS/A* tối ưu số bước, UCS

tối ưu tổng cost có phụ phí, còn DFS chỉ bảo đảm tìm lời giải hợp lệ trong điều kiện đã nêu. Độ dài và cost được trình bày tại Mục 6.4.

6.2 Peak memory

Trên 33 stage original, mỗi cặp BFS/DFS/UCS/A* có năm lượt AC với metrics cơ sở đầy đủ. Peak memory là đỉnh cấp phát Python do `tracemalloc` theo dõi, không phải RSS hay toàn bộ bộ nhớ tiến trình. Với A*, phép đo gồm tiền xử lý heuristic và tìm kiếm chính; không cộng peak các pha. Dùng MiB ($1\,\text{MiB} = 2^{20}$ byte); chi tiết phép đo tại Mục 5.3.

Median của 33 median peak memory theo stage là 228749 byte (xấp xỉ 0.218 MiB) với BFS, 160743 byte (0.153 MiB) với DFS, 219285 byte (0.209 MiB) với UCS và 261753 byte (0.250 MiB) với A*. Đây không phải peak của cả batch hay median của toàn bộ lượt gộp.

Bảng 13 minh họa bốn stage; mỗi ô ghi median (IQR), MiB. Dữ liệu đủ 33 stage được trình bày tại phụ lục.

Bảng 13: Peak memory minh họa theo stage: median (IQR), MiB

Stage	BFS	DFS	UCS	A*
08	0.221 (0.000)	0.139 (0.000)	0.217 (0.000)	1.213 (0.000)
15	1.699 (0.000)	0.326 (0.000)	1.858 (0.000)	4.334 (0.000)
23	1.568 (0.000)	3.198 (0.000)	1.623 (0.000)	7.357 (0.000)
26	3.811 (0.000)	1.641 (0.000)	3.434 (0.000)	3.024 (0.000)

IQR peak memory bằng đúng 0 byte ở cả 132 cặp thuật toán–stage, không chỉ bằng 0 sau khi làm tròn. Đây là độ phân tán quan sát trong batch, không bảo đảm bộ nhớ bất biến trên mọi môi trường. DFS có median peak thấp hơn BFS ở 24/33 stage, UCS thấp hơn BFS ở 22/33 và A* thấp hơn BFS ở 12/33; không thuật toán nào luôn dùng ít bộ nhớ nhất.

Stage23 cho thấy DFS có thể dùng nhiều bộ nhớ hơn BFS (3.198 so với 1.568 MiB), còn stage15 cho thấy UCS có thể cao hơn BFS (1.858 so với 1.699 MiB). Tại stage26, A* thấp hơn cả BFS và UCS (3.024 so với 3.811 và 3.434 MiB). Tiền xử lý heuristic là một thành phần bộ nhớ của A*, nhưng chưa có phép đo peak tách theo pha để xác định pha tạo đỉnh tại từng stage.

Peak phụ thuộc trạng thái được lưu, frontier và các cấu trúc hỗ trợ; không suy ra trực tiếp từ expanded nodes. DFS lưu visited và parent toàn cục nên không thể mặc định chỉ dùng bộ nhớ theo độ sâu. Kết quả mô tả cấp phát Python được theo dõi trong điều kiện thí nghiệm, không đại diện tổng RAM cần để chạy ứng dụng hoặc GUI.

6.3 Expanded nodes

Expanded nodes là số lần thực sự mở rộng trạng thái không phải đích để sinh successor, không phải số trạng thái được sinh hay số lần push/pop. UCS và A* bỏ bản ghi heap lỗi thời trước khi tăng counter; riêng A*, counter chỉ tính tìm kiếm chính, không tính tiền xử lý heuristic. Mỗi cặp thuật toán–stage có năm lượt AC và đủ metrics; cách đo và thống kê được trình bày tại Mục 5.3.

Median của 33 median expanded nodes theo stage là 474 với BFS, 282 với DFS, 493 với UCS và 265 với A*. Đây là median của các median theo stage, không phải tổng hoặc trung bình. Bảng 14 trình bày một số stage minh họa; mỗi ô ghi median (IQR), đơn vị trạng thái.

Bảng 14: Expanded nodes minh họa theo stage: median (IQR), trạng thái

Stage	BFS	DFS	UCS	A*
08	535 (0)	114 (0)	535 (0)	20 (0)
15	4304 (0)	518 (0)	4716 (0)	921 (0)
23	3916 (0)	5599 (0)	3990 (0)	3803 (0)
26	9830 (0)	3133 (0)	9205 (0)	5575 (0)

IQR bằng đúng 0 ở cả 132 cặp thuật toán–stage: counter lặp lại qua năm lượt trong batch này. Điều đó chỉ mô tả độ phân tán của expanded nodes trong batch, không cho thấy thời gian hoặc bộ nhớ cũng không biến thiên. Dữ liệu đủ 33 stage được trình bày tại phụ lục.

A* mở rộng ít trạng thái hơn cả BFS và UCS ở 33/33 stage. UCS mở rộng ít hơn BFS ở 4 stage, bằng ở 14 và nhiều hơn ở 15; DFS ít hơn BFS ở 27/33 stage. Đây chỉ là số đếm của pha tìm kiếm: BFS/A* tối ưu số action, UCS tối ưu tổng cost có phụ phí, còn DFS không bảo đảm tối ưu độ dài hoặc cost. Vì vậy, không thể dùng riêng expanded nodes để kết luận hiệu quả toàn diện. Số node cũng không suy ra trực tiếp thời gian hoặc bộ nhớ (Mục 6.1–6.2): A* còn tiền xử lý ngoài counter, và các thuật toán có chi phí xử lý trạng thái khác nhau.

6.4 Độ dài lời giải và chi phí

Độ dài $L = |\texttt{actions}|$ là số action của một lời giải thắng hợp lệ; C_U là weighted cost được tính lại theo Mục 3.6 cho lời giải của cả bốn thuật toán. BFS và A* tối ưu L, UCS tối ưu C_U, còn DFS không bảo đảm tối ưu đại lượng nào. Trường `cost` cũ của checker vẫn biểu diễn số bước, không thay cho `weighted_cost`. Mỗi cặp thuật toán–stage có năm lượt AC, đủ metrics và đối chiếu cost thành công; cách đo được trình bày tại Mục 5.3.

Median của 33 median theo stage là $(L, C_U) = (57, 68)$ với BFS, $(98, 110)$ với DFS, $(57, 68)$ với UCS và $(57, 68)$ với A*. Các median tổng quan này không hàm ý kết quả giống nhau trên từng stage. Bảng 15 minh họa cả hai đại lượng; mỗi ô ghi L/C_U, đều là median qua năm lượt. IQR của từng đại lượng bằng đúng 0 ở cả 132 cặp thuật toán–stage.

Bảng 15: Độ dài lời giải L / weighted cost C_U theo stage (median qua năm lượt)

Stage	**BFS**	**DFS**	**UCS**	**A***
08	10/13	98/101	10/13	10/13
13	46/68	57/85	49/63	46/68
15	57/70	186/207	57/70	57/70
16	28/56	70/100	30/55	28/56
23	75/133	748/1226	76/128	75/129
25	55/67	96/122	55/66	55/67
26	104/120	766/809	104/120	104/120

Dữ liệu đầy đủ 33 stage được trình bày tại phụ lục. BFS và A* có cùng độ dài ở 33/33 stage; DFS bằng BFS tại stage01 và dài hơn ở 32 stage còn lại. UCS bằng độ dài BFS/A* ở 30 stage, dài hơn tại stage13, stage16 và stage23; ở cả ba stage này, UCS có weighted cost thấp hơn. So với BFS và A*, UCS có weighted cost thấp hơn tại stage13, stage16, stage23 và stage25, bằng ở 29 stage còn lại. Stage25 cho thấy cùng độ dài vẫn có thể có cost khác nhau.

Tại stage23, BFS và A* đều có $L = 75$ nhưng cost lần lượt là 133 và 129; UCS chọn đường dài hơn một action với cost 128. Không thể kết luận BFS và A* luôn trả cùng chuỗi action hoặc cùng cost. AC xác nhận đường đi hợp lệ, còn replay xác nhận cost của đường đi chứ không tự chứng minh tối ưu toàn cục; bảo đảm tối ưu dựa trên thuật toán và mục tiêu tương ứng, với đối chiếu tối ưu riêng tại Mục 6.5. Vì vậy, cần xem xét cả L và C_U; thời gian hoặc expanded nodes thấp hơn không đủ để xếp hạng khi mục tiêu và chất lượng lời giải khác nhau.

6.5 Checker verdict

Thí nghiệm gồm 33 stage × 4 thuật toán × 5 lượt, tổng cộng 660 lượt. Bảng 16 tổng hợp verdict, metrics cơ sở và việc replay/tính lại C_U.

Bảng 16: Tổng hợp checker verdict và xác minh kết quả

Thuật toán	**AC/Tổng lượt**	**Đủ metrics cơ sở**	**Replay và tính lại C_U thành công**
BFS	165/165	165	165
DFS	165/165	165	165
UCS	165/165	165	165
A*	165/165	165	165

Cả 660/660 lượt đạt các tiêu chí trên; không có WA, RE hoặc TLE. Replay xác nhận chuỗi action tuân luật và đến đích, còn metrics đầy đủ là tiêu chí riêng. C_U được tính lại cho mọi lời giải theo Mục 3.6; riêng cả 165 lượt UCS đều có `metrics.weighted_cost` khớp giá trị này. Với BFS, DFS và A*, cost được tính ở bước đánh giá, không hàm ý các solver này tự xuất trường weighted cost. Cost khớp xác nhận giá trị của đường đi, không chứng minh không tồn tại đường rẻ hơn. BFS/A* còn được đối chiếu số bước với mốc tham chiếu; DFS và UCS không bị từ chối chỉ vì lời giải dài hơn. Tiêu chí kiểm tra được trình bày tại Mục 5.2–5.3.

Ngoài benchmark, một cài đặt Dijkstra riêng trên mô hình checker, dùng cách tính cost riêng và không gọi hàm tìm kiếm hoặc cost của UCS, được dùng để đối chiếu trên 33 stage original. Cost UCS khớp giá trị tối ưu đối chiếu ở cả 33 stage; cả năm lượt UCS trên mỗi stage đều đạt giá trị đó. Phép đối chiếu này nằm ngoài cửa sổ đo thời gian/bộ nhớ, là kiểm chứng bổ sung trên bộ test đang xét chứ không phải verdict AC hay chứng minh cho mọi màn tùy ý. Bảo đảm lý thuyết của UCS được trình bày tại Mục 4.7.

6.6 Thảo luận kết quả

AC xác nhận lời giải hợp lệ, không tự đánh giá đầy đủ chất lượng tìm kiếm. Trong batch này, cả bốn thuật toán đều trả về lời giải hợp lệ, đủ metrics và có cost tính lại thành công (Mục 6.5). BFS và A* tối ưu số action L, UCS tối ưu cost có phụ phí C_U theo Mục 3.6, còn DFS không bảo đảm tối ưu hai mục tiêu. Vì vậy, cần xem kết quả tài nguyên cùng chất lượng đường đi thay vì xếp hạng bằng một chỉ tiêu riêng.

BFS là mốc đối chiếu phù hợp khi ưu tiên lời giải ít bước. BFS và A* có cùng độ dài trên mọi stage, nhưng không nhất thiết trả cùng đường đi hoặc weighted cost (Mục 6.4). DFS thường tìm lời giải nhanh hơn BFS trong batch, song thường trả đường dài hơn và có weighted cost cao hơn. Điều này không có nghĩa DFS luôn nhanh hơn hoặc dùng ít bộ nhớ hơn; thí nghiệm đo thời gian tìm kiếm,

không đo thời gian người chơi hay GUI thực hiện toàn bộ lời giải (Mục 6.1–6.2).

UCS làm rõ khác biệt giữa ít bước và ít cost: ở một số stage, thuật toán chọn đường dài hơn để giảm C_U, hoặc chọn đường khác cùng độ dài nhưng rẻ hơn (Mục 6.4). Các trọng số tại Mục 3.6 biểu đạt ưu tiên thiết kế của nhóm, không phải chi phí vật lý hay xác suất thua. Việc tính lại C_U cho BFS, DFS và A* giúp so sánh đường đi theo cùng thước đo nhưng không đổi mục tiêu tìm kiếm của các thuật toán này.

A* mở rộng ít trạng thái hơn BFS và UCS trên bộ test, nhưng tổng thời gian và peak memory không phải lúc nào cũng thấp hơn (Mục 6.1–6.3). Tiền xử lý heuristic tiêu tốn tài nguyên và không nằm trong counter expanded nodes; vì vậy hai kết quả không mâu thuẫn. Chưa có phép đo peak tách theo pha để xác định chính xác pha tạo đỉnh bộ nhớ ở từng stage.

Các nhận xét chỉ áp dụng cho bộ test và môi trường hiện tại. IQR thời gian lớn ở một số stage đòi hỏi thận trọng khi so sánh thứ hạng; đây không phải kiểm định ưu thế thống kê (Mục 6.1). Peak memory là cấp phát Python được `tracemalloc` theo dõi, không phải RSS (Mục 6.2). Theo mục tiêu đã chọn, có thể cân nhắc BFS/A* khi cần ít bước, UCS khi cần tối thiểu hóa C_U, hoặc DFS khi cần một đường thắng và chấp nhận chất lượng lời giải thấp hơn.

7 Hạn chế và hướng phát triển

7.1 Hạn chế hiện tại

Đánh giá giới hạn ở 33 stage original cố định theo Mục 5.2; sáu sample và unit tests bổ sung kiểm tra nhưng chưa bao phủ mọi bản đồ, tổ hợp cơ chế hoặc tương tác GUI. Thí nghiệm chạy trên một môi trường với năm lượt mỗi cặp; IQR thời gian lớn ở một số stage nên chưa đủ kết luận thứ hạng ổn định hay ưu thế thống kê (Mục 6.1). Việc các thuật toán tối ưu những mục tiêu khác nhau là đặc điểm cần lưu ý khi so sánh, không phải lỗi triển khai.

Thời gian chịu overhead instrumentation; `tracemalloc` chỉ theo dõi cấp phát Python, không phải RSS. Chưa đo chi phí GUI/animation hoặc peak memory tách theo pha A*. Cost UCS đã được định nghĩa và kiểm chứng trên bộ test, nhưng trọng số phản ánh ưu tiên do nhóm lựa chọn; chưa khảo sát độ nhạy khi thay trọng số hay mức phù hợp trong mọi tình huống sử dụng (Mục 3.6).

7.2 Hướng phát triển

Các hướng sau chưa được đo trong batch hiện tại.

Mở rộng đánh giá với trường hợp biên, tổ hợp cơ chế và kiểm thử GUI, đồng thời chạy trên nhiều phiên và môi trường để khảo sát dao động thời gian. Bản đồ tự tạo cần được xác thực cấu trúc và xác nhận có lời giải trước khi đưa vào tập đánh giá.

Thay đổi trọng số UCS có hệ thống, đánh giá ảnh hưởng đến L, C_U và tài nguyên tìm kiếm. Chưa thể kết luận bộ trọng số tốt nhất nếu chưa thử nghiệm.

Khảo sát heuristic A* nhẹ hơn, giảm tiền xử lý hoặc tái sử dụng bảng khi giải lặp cùng màn. Nếu xây dựng A* tối ưu C_U, cần đổi g sang cost có phụ phí và xác nhận điều kiện heuristic tương ứng; đây là cấu hình khác A* hiện tại tối ưu số bước.

7.3 Ghi nhận nguồn và quyền sử dụng

Đề bài và FAQ môn học là tài liệu nội bộ nhóm được cung cấp[8, 1]. Kho `tkoz0/bloxorz-solver` cung cấp dữ liệu mã hóa 33 stage và lời giải tham chiếu; stage15 được điều chỉnh theo luật checker, và kho không nêu giấy phép rõ ràng trong bản rà soát[7]. Trang Coolmath Games ghi nhận Damien Clarke là nhà phát triển Bloxorz[2]. Template báo cáo dựa trên template HCMUS không chính thức, có giấy phép GPL-3.0[3]; giấy phép template không mặc nhiên áp dụng cho logo hoặc tài nguyên ngoài template. Chưa có thông tin giấy phép riêng cho một số tài nguyên hình ảnh.

Tài liệu tham khảo

[1] AI 2627 HK1 — Project FAQs — Project 1. Tài liệu hỏi đáp môn học. Tài liệu nội bộ được cung cấp.

[2] Coolmath Games. Bloxorz. https://www.coolmathgames.com/0-bloxorz. Trang ghi nhà phát triển Damien Clarke; truy cập ngày 31/10/2026.

[3] khongsomeo and contributors. HCMUS Unofficial Report Template. https://github.com/khongsomeo/hcmus-unofficial-report-template. Kho công bố giấy phép GPL-3.0; truy cập ngày 31/10/2026.

[4] Panda3D. Panda3D Manual: Python. https://docs.panda3d.org/1.10/python/index. Nhánh tài liệu 1.10; phiên bản sử dụng: 1.10.16; truy cập ngày 31/10/2026.

[5] Pillow. Pillow (PIL Fork) Documentation. https://pillow.readthedocs.io/en/stable/. Phiên bản sử dụng: 13.0.0; truy cập ngày 31/10/2026.

[6] Python Software Foundation. Python 3.12 Documentation. https://docs.python.org/3.12/library/tracemalloc.html; https://docs.python.org/3.12/library/time.html#time.perf_counter_ns; https://docs.python.org/3.12/library/statistics.html#statistics.quantiles. Các mục tracemalloc, time.perf_counter_ns và statistics.quantiles; truy cập ngày 31/10/2026.

[7] tkoz0. bloxorz-solver. https://github.com/tkoz0/bloxorz-solver. Nguồn dữ liệu mã hóa 33 stage và lời giải tham chiếu; truy cập ngày 31/10/2026.

[8] Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM. Project 1 — Bloxorz Solver. Đề bài nhóm, môn CSC14003 / Introduction to Artificial Intelligence. Tài liệu nội bộ được cung cấp.

[9] University of California, Berkeley, CS188. Introduction to Artificial Intelligence: 1.3 Uninformed Search. https://inst.eecs.berkeley.edu/~cs188/textbook/search/uninformed.html. Truy cập ngày 31/10/2026.

[10] University of California, Berkeley, CS188. Introduction to Artificial Intelligence: 1.4 Informed Search. https://inst.eecs.berkeley.edu/~cs188/textbook/search/informed.html. Truy cập ngày 31/10/2026.

[11] Ursina. Ursina Documentation. https://www.ursinaengine.org/. Phiên bản sử dụng: 8.3.0; truy cập ngày 31/10/2026.

A Cài đặt và thực thi

A.1 Báo cáo LaTeX

Báo cáo được biên dịch từ thư mục gốc bằng:

```
latexmk -pdf main.tex
```

A.2 Engine và solver

Các lệnh chạy từ thư mục gốc repository hoặc `Source` trong gói nộp. Solver, checker và terminal dùng thư viện chuẩn, tương thích Python $\geq$ 3.10. GUI yêu cầu Python $\geq$ 3.12 (đã kiểm tra với Python 3.12.3; Ursina 8.3.0 yêu cầu Python $\geq$ 3.12). Runtime được ghim ở Ursina 8.3.0, Panda3D 1.10.16 và Pillow 13.0.0. Dependencies vẽ biểu đồ là tùy chọn, không cần cài để chơi hoặc giải.

Cài đặt GUI trên Linux/macOS:

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

Trên Windows PowerShell, thay dòng kích hoạt bằng `.venv\Scripts\Activate.ps1`. Thư mục `.venv` và tiến độ chơi là dữ liệu cục bộ, không nằm trong gói mã nguồn.

Ví dụ chơi, giải màn, kiểm tra và replay:

```
python play.py
python play.py --stage 13
python play.py --suite samples --stage 04
python play.py --stage 01 --terminal
python algorithms/UCS.py < testcases/original/13.in.json > answer.json
python checker.py --input testcases/original/13.in.json --output answer.json
python play.py --stage 13 --replay answer.json
python play.py --stage 13 --terminal --replay answer.json --delay 0.2
python -m unittest discover -s tests -v
python tools/validate_solvers.py
```

Có thể thay `UCS.py` bằng `BFS.py`, `DFS.py` hoặc `ASTAR.py` để chạy solver tương ứng trên cùng stage. Không đổi stage đầu vào mà giữ file `answer.json` của stage khác.

Chạy bộ original:

```
python run_tests.py --algorithm algorithms/BFS.py --all --require-optimal
python run_tests.py --algorithm algorithms/ASTAR.py --all --require-optimal
python run_tests.py --algorithm algorithms/DFS.py --all
python run_tests.py --algorithm algorithms/UCS.py --all
```

Thêm `-suite samples` để kiểm tra sáu mẫu. `tools/validate_solvers.py` kiểm tra bốn thuật toán trên cả 39 testcase và đối chiếu UCS với Dijkstra riêng trên mô hình checker. Kết quả kiểm tra phiên bản tích hợp hiện tại là 156/156 lượt đạt, không phải batch benchmark 660 lượt.

BFS/A* tối ưu số action; UCS tối ưu weighted cost theo Mục 3.6. Với UCS, không dùng cờ `--require-optimal`, vì cờ này chỉ so số bước tham chiếu. `cost/solution_length` của checker vẫn là số action; solver UCS có `metrics.weighted_cost`. Stage 13 có 49 bước và weighted cost 63. AC chỉ xác nhận lời giải hợp lệ.

A.3 Hướng dẫn vận hành giao diện

Lệnh cài đặt và khởi chạy xem A.2. Giao diện cho phép chọn trực tiếp A*, BFS, DFS hoặc UCS. Nút Giải chạy solver; lời giải hợp lệ sẽ tự replay[1]. Khi đang tìm kiếm, nút Giải chuyển thành Tạm Dừng/Tiếp Tục.

Điều khiển. WASD hoặc phím mũi tên để di chuyển; Space đổi cube; Z/U Undo; R Reset; Esc mở menu.

Camera và replay. Camera trực giao: con lăn để zoom, C trả camera về mặc định; không có thao tác kéo chuột phải để xoay hoặc phím V đổi phối cảnh. Replay có Prev, Play/Pause, Next và điều chỉnh tốc độ; P tạm dừng/tiếp tục replay, không có nút Start/End.

Benchmark. Cửa sổ benchmark chạy bốn thuật toán; Run phát lời giải trên stage tương ứng. Cột STEPS/COST hiển thị riêng số bước/cost của UCS (ví dụ 49/63), như HUD cũng phân biệt hai đại lượng. Giao diện phù hợp nhất với tỉ lệ cửa sổ 16:9 để thấy đầy đủ HUD.

B Lệnh benchmark và kết quả chi tiết

B.1 Lệnh chạy

Lệnh tái lập cấu hình benchmark; thư mục output phải là thư mục mới. Log là dữ liệu cục bộ.

```
python tools/benchmark.py --algorithms BFS DFS UCS ASTAR --suite original --
   repeats 5 --timeout 60 --seed 20261028 --output results/benchmark-new
```

Bộ số liệu công bố được thu ngày 28/10/2026; chạy lệnh ở thời điểm khác tạo batch mới và không bảo đảm có cùng thời gian.

B.2 Kết quả theo stage

Benchmark gồm 33 stage original, bốn thuật toán và năm lượt mỗi cặp: 132 cặp, tổng cộng 660 lượt chính. Các lượt chạy tuần tự, mỗi lượt dùng tiến trình Python mới; công việc được xáo trộn mỗi vòng với seed 20261028, timeout 60 giây. Warmup sample01 một lượt mỗi solver và 24 lượt kiểm tra sáu sample nằm ngoài 660 lượt.

Máy chạy: Ryzen 7 8845H, 16 CPU logic, khoảng 27,21 GiB RAM; CPython 3.12.3, Linux x86_64, kernel 7.0.0-38.

Mỗi ô ghi median (IQR) qua năm lượt; IQR $= Q_3 - Q_1$, dùng phương pháp inclusive. Thời gian và peak memory của A* gồm tiền xử lý heuristic; expanded nodes chỉ tính tìm kiếm chính. Peak memory là cấp phát Python theo `tracemalloc`, không phải RSS. Độ dài L và weighted cost C_U là hai chỉ tiêu riêng: BFS/A* tối ưu L, UCS tối ưu C_U. Cost dùng chung công thức $1 + 1\times$ chạm B $+2\times$ chạm F $+3\times$ kích hoạt tách. Replay và kiểm tra nằm ngoài phép đo search. IQR của memory, expanded nodes, số bước và cost đều bằng 0; IQR thời gian được giữ theo số liệu đo.

Mỗi thuật toán đạt 165/165 AC, đủ metrics và tính lại weighted cost thành công. Cost tự báo cáo của UCS khớp replay ở 165/165 lượt; đối chiếu riêng bằng Dijkstra trên checker khớp 33/33 stage. AC và cost replay đúng không tự chứng minh tính tối ưu.

Bảng 17: Thời gian giải theo stage: median (IQR), ms

Stage	**BFS**	**DFS**	**UCS**	**A***
Stage01	4.458 (0.469)	5.270 (1.006)	4.552 (0.660)	7.991 (0.934)

Tiếp tục ở trang sau

Bảng 17 (tiếp theo)

Stage	**BFS**	**DFS**	**UCS**	**A***
Stage02	19.585 (0.326)	18.845 (0.829)	19.892 (0.519)	19.438 (0.351)
Stage03	5.881 (0.252)	2.853 (0.360)	6.038 (0.091)	11.098 (0.934)
Stage04	4.886 (0.725)	5.433 (0.381)	4.879 (0.209)	12.510 (0.045)
Stage05	21.599 (25.971)	9.609 (0.461)	25.977 (136.928)	22.994 (0.547)
Stage06	6.956 (0.497)	4.935 (0.124)	7.150 (59.207)	12.143 (1.204)
Stage07	12.782 (1.189)	10.576 (3.538)	13.214 (1.158)	15.263 (1.138)
Stage08	49.447 (4.227)	9.877 (0.260)	55.632 (0.688)	151.470 (3.451)
Stage09	319.124 (494.741)	84.590 (3.761)	81.687 (123.329)	175.599 (11.102)
Stage10	1193.197 (585.195)	81.811 (4.391)	1465.939 (1765.307)	780.939 (1282.186)
Stage11	12.742 (0.780)	11.811 (2.652)	12.636 (2.328)	16.769 (0.655)
Stage12	22.629 (1.557)	20.837 (3.369)	22.265 (1.500)	27.882 (1.300)
Stage13	8.619 (0.502)	8.478 (0.985)	9.007 (0.325)	23.513 (5.070)
Stage14	29.007 (3.027)	14.502 (0.833)	33.528 (1.504)	33.841 (1.815)
Stage15	1417.523 (1198.450)	50.043 (13.366)	582.693 (19.799)	692.192 (2.641)
Stage16	13.046 (0.297)	36.107 (1.927)	16.169 (0.782)	48.985 (1.391)
Stage17	63.992 (3.562)	45.778 (197.559)	72.825 (0.848)	71.013 (150.145)
Stage18	41.449 (2.157)	27.411 (2.143)	44.502 (1.118)	48.454 (0.646)
Stage19	14.513 (1.152)	10.158 (0.815)	15.493 (0.665)	18.568 (0.391)
Stage20	1619.001 (561.133)	316.978 (11.823)	472.121 (580.786)	613.514 (45.382)
Stage21	16.116 (2.079)	9.324 (0.274)	16.544 (1.105)	16.831 (0.839)
Stage22	21.976 (1.235)	14.766 (0.514)	71.628 (189.302)	30.081 (2.081)
Stage23	400.831 (589.835)	585.136 (233.222)	461.285 (26.663)	3306.104 (2053.381)
Stage24	35.789 (2.855)	9.465 (75.791)	40.015 (1.446)	427.727 (619.940)
Stage25	30.579 (1.684)	22.032 (1.961)	34.321 (1.446)	29.149 (0.975)
Stage26	1673.590 (584.975)	323.695 (7.677)	1016.199 (31.980)	1773.977 (2347.786)
Stage27	83.890 (91.396)	17.389 (0.450)	23.576 (0.895)	34.251 (2.089)
Stage28	686.010 (10.129)	1498.561 (982.021)	752.328 (383.045)	700.723 (26.897)
Stage29	146.838 (2.738)	84.598 (226.451)	164.817 (26.977)	162.118 (8.662)
Stage30	37.944 (0.483)	18.758 (0.198)	41.364 (1.595)	47.182 (3.811)

Tiếp tục ở trang sau

Bảng 17 (tiếp theo)

Stage	**BFS**	**DFS**	**UCS**	**A***
Stage31	58.447 (94.020)	34.418 (1.309)	65.714 (2.938)	68.905 (10.751)
Stage32	41.154 (2.000)	28.702 (1.755)	43.399 (2.623)	49.039 (0.959)
Stage33	50.425 (0.982)	48.178 (206.010)	55.143 (0.132)	72.447 (2.364)

Bảng 18: Peak memory theo stage: median (IQR), MiB

Stage	**BFS**	**DFS**	**UCS**	**A***
Stage01	0.041 (0.000)	0.044 (0.000)	0.035 (0.000)	0.071 (0.000)
Stage02	0.151 (0.000)	0.153 (0.000)	0.121 (0.000)	0.108 (0.000)
Stage03	0.051 (0.000)	0.034 (0.000)	0.047 (0.000)	0.090 (0.000)
Stage04	0.039 (0.000)	0.052 (0.000)	0.039 (0.000)	0.104 (0.000)
Stage05	0.158 (0.000)	0.090 (0.000)	0.133 (0.000)	0.131 (0.000)
Stage06	0.057 (0.000)	0.051 (0.000)	0.053 (0.000)	0.096 (0.000)
Stage07	0.085 (0.000)	0.084 (0.000)	0.086 (0.000)	0.095 (0.000)
Stage08	0.221 (0.000)	0.139 (0.000)	0.217 (0.000)	1.213 (0.000)
Stage09	0.269 (0.000)	0.574 (0.000)	0.279 (0.000)	1.369 (0.000)
Stage10	2.639 (0.000)	0.767 (0.000)	2.470 (0.000)	2.542 (0.000)
Stage11	0.086 (0.000)	0.084 (0.000)	0.087 (0.000)	0.091 (0.000)
Stage12	0.155 (0.000)	0.172 (0.000)	0.133 (0.000)	0.143 (0.000)
Stage13	0.063 (0.000)	0.065 (0.000)	0.060 (0.000)	0.139 (0.000)
Stage14	0.199 (0.000)	0.104 (0.000)	0.186 (0.000)	0.195 (0.000)
Stage15	1.699 (0.000)	0.326 (0.000)	1.858 (0.000)	4.334 (0.000)
Stage16	0.079 (0.000)	0.199 (0.000)	0.092 (0.000)	0.329 (0.000)
Stage17	0.383 (0.000)	0.344 (0.000)	0.389 (0.000)	0.361 (0.000)
Stage18	0.249 (0.000)	0.210 (0.000)	0.237 (0.000)	0.255 (0.000)
Stage19	0.101 (0.000)	0.080 (0.000)	0.101 (0.000)	0.103 (0.000)
Stage20	1.668 (0.000)	1.343 (0.000)	1.659 (0.000)	3.770 (0.000)
Stage21	0.104 (0.000)	0.075 (0.000)	0.104 (0.000)	0.097 (0.000)
Stage22	0.177 (0.000)	0.113 (0.000)	0.163 (0.000)	0.150 (0.000)

Tiếp tục ở trang sau

Bảng 18 (tiếp theo)

Stage	**BFS**	**DFS**	**UCS**	**A***
Stage23	1.568 (0.000)	3.198 (0.000)	1.623 (0.000)	7.357 (0.000)
Stage24	0.225 (0.000)	0.076 (0.000)	0.214 (0.000)	2.900 (0.000)
Stage25	0.218 (0.000)	0.181 (0.000)	0.209 (0.000)	0.157 (0.000)
Stage26	3.811 (0.000)	1.641 (0.000)	3.434 (0.000)	3.024 (0.000)
Stage27	0.155 (0.000)	0.112 (0.000)	0.133 (0.000)	0.162 (0.000)
Stage28	2.798 (0.000)	3.054 (0.000)	2.658 (0.000)	3.879 (0.000)
Stage29	0.990 (0.000)	0.624 (0.000)	0.938 (0.000)	0.924 (0.000)
Stage30	0.242 (0.000)	0.130 (0.000)	0.228 (0.000)	0.250 (0.000)
Stage31	0.368 (0.000)	0.253 (0.000)	0.387 (0.000)	0.382 (0.000)
Stage32	0.259 (0.000)	0.210 (0.000)	0.247 (0.000)	0.258 (0.000)
Stage33	0.308 (0.000)	0.305 (0.000)	0.311 (0.000)	0.373 (0.000)

Bảng 19: Expanded nodes theo stage: median (IQR), trạng thái

Stage	**BFS**	**DFS**	**UCS**	**A***
Stage01	64 (0)	82 (0)	64 (0)	14 (0)
Stage02	283 (0)	282 (0)	269 (0)	93 (0)
Stage03	87 (0)	42 (0)	87 (0)	26 (0)
Stage04	69 (0)	83 (0)	69 (0)	28 (0)
Stage05	319 (0)	135 (0)	312 (0)	138 (0)
Stage06	106 (0)	77 (0)	106 (0)	46 (0)
Stage07	177 (0)	153 (0)	177 (0)	93 (0)
Stage08	535 (0)	114 (0)	535 (0)	20 (0)
Stage09	734 (0)	862 (0)	734 (0)	85 (0)
Stage10	5883 (0)	803 (0)	5977 (0)	3815 (0)
Stage11	186 (0)	168 (0)	186 (0)	124 (0)
Stage12	320 (0)	306 (0)	324 (0)	250 (0)
Stage13	134 (0)	129 (0)	135 (0)	123 (0)

Tiếp tục ở trang sau

Bảng 19 (tiếp theo)

Stage	**BFS**	**DFS**	**UCS**	**A***
Stage14	459 (0)	209 (0)	459 (0)	334 (0)
Stage15	4304 (0)	518 (0)	4716 (0)	921 (0)
Stage16	151 (0)	381 (0)	170 (0)	69 (0)
Stage17	983 (0)	658 (0)	987 (0)	767 (0)
Stage18	598 (0)	392 (0)	598 (0)	547 (0)
Stage19	223 (0)	151 (0)	223 (0)	129 (0)
Stage20	4277 (0)	3182 (0)	4159 (0)	1129 (0)
Stage21	234 (0)	143 (0)	234 (0)	129 (0)
Stage22	333 (0)	215 (0)	333 (0)	270 (0)
Stage23	3916 (0)	5599 (0)	3990 (0)	3803 (0)
Stage24	520 (0)	125 (0)	534 (0)	420 (0)
Stage25	474 (0)	331 (0)	493 (0)	262 (0)
Stage26	9830 (0)	3133 (0)	9205 (0)	5575 (0)
Stage27	314 (0)	255 (0)	333 (0)	265 (0)
Stage28	6606 (0)	7038 (0)	6659 (0)	1680 (0)
Stage29	2298 (0)	1250 (0)	2301 (0)	2101 (0)
Stage30	556 (0)	274 (0)	556 (0)	462 (0)
Stage31	872 (0)	513 (0)	912 (0)	743 (0)
Stage32	630 (0)	435 (0)	633 (0)	553 (0)
Stage33	737 (0)	712 (0)	738 (0)	669 (0)

Bảng 20: Độ dài lời giải theo stage: median (IQR), action

Stage	**BFS**	**DFS**	**UCS**	**A***
Stage01	7 (0)	7 (0)	7 (0)	7 (0)
Stage02	17 (0)	68 (0)	17 (0)	17 (0)
Stage03	19 (0)	38 (0)	19 (0)	19 (0)
Stage04	28 (0)	39 (0)	28 (0)	28 (0)
Stage05	33 (0)	91 (0)	33 (0)	33 (0)

Tiếp tục ở trang sau

Bảng 20 (tiếp theo)

Stage	**BFS**	**DFS**	**UCS**	**A***
Stage06	35 (0)	51 (0)	35 (0)	35 (0)
Stage07	44 (0)	69 (0)	44 (0)	44 (0)
Stage08	10 (0)	98 (0)	10 (0)	10 (0)
Stage09	24 (0)	276 (0)	24 (0)	24 (0)
Stage10	57 (0)	522 (0)	57 (0)	57 (0)
Stage11	47 (0)	49 (0)	47 (0)	47 (0)
Stage12	65 (0)	113 (0)	65 (0)	65 (0)
Stage13	46 (0)	57 (0)	49 (0)	46 (0)
Stage14	67 (0)	106 (0)	67 (0)	67 (0)
Stage15	57 (0)	186 (0)	57 (0)	57 (0)
Stage16	28 (0)	70 (0)	30 (0)	28 (0)
Stage17	106 (0)	315 (0)	106 (0)	106 (0)
Stage18	85 (0)	168 (0)	85 (0)	85 (0)
Stage19	67 (0)	87 (0)	67 (0)	67 (0)
Stage20	56 (0)	209 (0)	56 (0)	56 (0)
Stage21	71 (0)	72 (0)	71 (0)	71 (0)
Stage22	65 (0)	124 (0)	65 (0)	65 (0)
Stage23	75 (0)	748 (0)	76 (0)	75 (0)
Stage24	57 (0)	81 (0)	57 (0)	57 (0)
Stage25	55 (0)	96 (0)	55 (0)	55 (0)
Stage26	104 (0)	766 (0)	104 (0)	104 (0)
Stage27	71 (0)	77 (0)	71 (0)	71 (0)
Stage28	100 (0)	374 (0)	100 (0)	100 (0)
Stage29	104 (0)	132 (0)	104 (0)	104 (0)
Stage30	114 (0)	130 (0)	114 (0)	114 (0)
Stage31	91 (0)	187 (0)	91 (0)	91 (0)
Stage32	129 (0)	177 (0)	129 (0)	129 (0)
Stage33	65 (0)	79 (0)	65 (0)	65 (0)

Bảng 21: Weighted cost C_U theo stage: median (IQR)

Stage	**BFS**	**DFS**	**UCS**	**A***
Stage01	7 (0)	7 (0)	7 (0)	7 (0)
Stage02	20 (0)	72 (0)	20 (0)	20 (0)
Stage03	19 (0)	38 (0)	19 (0)	19 (0)
Stage04	56 (0)	67 (0)	56 (0)	56 (0)
Stage05	36 (0)	96 (0)	36 (0)	36 (0)
Stage06	35 (0)	51 (0)	35 (0)	35 (0)
Stage07	45 (0)	70 (0)	45 (0)	45 (0)
Stage08	13 (0)	101 (0)	13 (0)	13 (0)
Stage09	27 (0)	279 (0)	27 (0)	27 (0)
Stage10	66 (0)	547 (0)	66 (0)	66 (0)
Stage11	49 (0)	51 (0)	49 (0)	49 (0)
Stage12	68 (0)	117 (0)	68 (0)	68 (0)
Stage13	68 (0)	85 (0)	63 (0)	68 (0)
Stage14	69 (0)	108 (0)	69 (0)	69 (0)
Stage15	70 (0)	207 (0)	70 (0)	70 (0)
Stage16	56 (0)	100 (0)	55 (0)	56 (0)
Stage17	113 (0)	324 (0)	113 (0)	113 (0)
Stage18	93 (0)	178 (0)	93 (0)	93 (0)
Stage19	73 (0)	93 (0)	73 (0)	73 (0)
Stage20	66 (0)	228 (0)	66 (0)	66 (0)
Stage21	73 (0)	74 (0)	73 (0)	73 (0)
Stage22	70 (0)	129 (0)	70 (0)	70 (0)
Stage23	133 (0)	1226 (0)	128 (0)	129 (0)
Stage24	68 (0)	92 (0)	68 (0)	68 (0)
Stage25	67 (0)	122 (0)	66 (0)	67 (0)
Stage26	120 (0)	809 (0)	120 (0)	120 (0)
Stage27	104 (0)	110 (0)	104 (0)	104 (0)
Stage28	121 (0)	445 (0)	121 (0)	121 (0)

Tiếp tục ở trang sau

Bảng 21 (tiếp theo)

Stage	BFS	DFS	UCS	A*
Stage29	122 (0)	150 (0)	122 (0)	122 (0)
Stage30	250 (0)	286 (0)	250 (0)	250 (0)
Stage31	114 (0)	235 (0)	114 (0)	114 (0)
Stage32	135 (0)	183 (0)	135 (0)	135 (0)
Stage33	69 (0)	83 (0)	69 (0)	69 (0)

C Ma trận kiểm thử giao diện và engine

C.1 Ma trận 19 ca kiểm thử

Các luật chơi, cơ chế tương tác và độ an toàn của giao diện được kiểm tra bằng bộ test trong `tests/test_rules_consistency.py`; báo cáo tự sinh khi chạy test được xuất ra `results/tests/test_matrix_report.json` và không được Git theo dõi. Cả 19 ca đều đạt (19/19 `PASSED`), vượt mức tối thiểu 10 testcase nêu trong FAQ[1]. Bảng 22 nhóm các ca theo cơ chế; đây là kiểm thử bổ sung trên một số màn đại diện, không phải bảo đảm bao phủ mọi tổ hợp cơ chế.

Bảng 22: Ma trận kiểm thử: kịch bản và kết quả (tất cả `PASSED`)

ID	Nhóm	Ca kiểm thử	Kịch bản	Dữ liệu	Kết quả
P01	Tư thế, rơi	`test_P01_posture_rolling_cycle`	Khối $1 \times 1 \times 2$ lăn từ đứng sang nằm rồi về đứng.	`samples/01.in.json`	Tọa độ và tư thế của replay khớp engine.
P02	Tư thế, rơi	`test_P02_fall_off_grid`	Khối lăn quá mép sàn.	Stage 01, action `L`	Ván thua, lý do là rơi khỏi mép sàn.
F01	Dễ vỡ, đích	`test_F01_fragile_standing_collapse`	Khối đứng thẳng trên ô `F`.	`samples/03.in.json`	Ô vỡ; engine và replay cùng báo thua.
F02	Dễ vỡ, đích	`test_F02_fragile_horizontal_safe`	Khối nằm đè lên ô `F`.	`samples/03.in.json`	Đi qua an toàn, không thua.
F03	Dễ vỡ, đích	`test_F03_goal_horizontal_not_win`	Khối nằm qua ô đích.	Stage 01 biến thể	Không thắng; phải đứng thẳng.
S01	Công tắc, cầu	`test_S01_soft_switch_toggles_bridge`	Một phần khối chạm công tắc nhẹ `O`.	Stage 02	Cầu đổi trạng thái đúng cấu hình.

Tiếp tục ở trang sau

Bảng 22 (tiếp theo)

ID	Nhóm	Ca kiểm thử	Kịch bản	Dữ liệu	Kết quả
S02	Công tắc, cầu	`test_S02_hard_switch_requires_standing`	Khối nằm qua công tắc nặng `X`.	Stage 04	Cầu giữ nguyên; chỉ kích hoạt khi đứng thẳng.
S03	Công tắc, cầu	`test_S03_stationary_cube_on_switch`	Cube đứng yên trên công tắc khi cube kia di chuyển.	Stage 08 (có tách)	Công tắc chỉ kích hoạt khi bước vào ô.
S04	Công tắc, cầu	`test_S04_bridge_collapse_under_stationary_cube`	Cầu tắt dưới chân cube đứng yên.	Stage 08 biến thể	Phát hiện mất chỗ đỡ, xử lý rơi ngay.
T01	Tách, ghép	`test_T01_splitter_activates_on_standing`	Khối đứng thẳng vào ô `T`.	`original/08.in.json`	`is_split=True`, hai cube tại hai điểm đích.
T02	Tách, ghép	`test_T02_switch_active_cube`	Đổi cube điều khiển bằng `Space`.	Stage 08	Đổi cube đúng, cube kia giữ tọa độ.
T03	Tách, ghép	`test_T03_merge_cubes_when_adjacent`	Hai cube lăn đến vị trí kề cạnh.	Stage 08	Tự ghép về khối $1 \times 1 \times 2$, hết trạng thái tách.
M01	Reset, đổi màn	`test_M01_reset_restores_initial_state`	Reset giữa ván.	Stage 01, 02	Khôi phục bàn chơi, ô dễ vỡ, số bước về 0.
M02	Reset, đổi màn	`test_M02_stage_switching_loads_catalog`	Đổi màn từ danh mục.	Danh mục (Stage 01, 02)	Nạp đúng bàn chơi mới, hủy tiến trình cũ.
E01	Lỗi, timeout	`test_E01_corrupt_solver_output_rejected_safely`	Solver trả dữ liệu sai định dạng (JSON hỏng, thiếu trường).	Dữ liệu lỗi giả lập	Checker bắt lỗi an toàn, verdict `RE`.
E02	Lỗi, timeout	`test_E02_illegal_solution_rejected_by_checker`	Lời giải có nước đi sai luật.	Action `L, L`	Checker từ chối, verdict `WA`.
E03	Lỗi, timeout	`test_E03_search_timeout_handled`	Solver vượt thời gian cho phép.	Stage 10, timeout 0.001 s	Chuyển sang trạng thái `TIMEOUT`, thông báo an toàn.

Tiếp tục ở trang sau

Bảng 22 (tiếp theo)

ID	Nhóm	Ca kiểm thử	Kịch bản	Dữ liệu	Kết quả
R01	Tạm dừng solver	`test_R01_search_pause_and_resume`	Tạm dừng và tiếp tục solver nhiều lần.	Stage 10 (BFS)	Giải xong với `AC`; số nút giữ nguyên trong lúc dừng.
R02	Tạm dừng solver	`test_R02_search_cancel_on_stage_change`	Đổi màn khi solver đang tạm dừng.	Stage 10	Luồng nền được giải phóng ngay.

C.2 Trace đối chiếu ba bên

Lớp `ThreeWayHarness` so khớp trạng thái của replay trong checker, engine và giao diện sau từng action. Dưới đây là trích trace trên Stage 01 với lời giải 7 bước `R, R, D, R, R, R, D`; các bước 4 đến 6 được lược bớt (theo nhóm GUI đều khớp).

```
[TRACE START] Stage: stage01 | Start Cell: (1, 1) | Posture: STANDING
Step 1: Action 'R'
  Replay: Cells {(1, 2), (1, 3)} | HORIZONTAL_Y | Split: False | Won: False | Lost: False
  Engine: Cells {(1, 2), (1, 3)} | HORIZONTAL_Y | Split: False | Won: False | Lost: False
  => MATCH
Step 2: Action 'R'
  Replay: Cells {(1, 4)}         | STANDING     | Split: False | Won: False | Lost: False
  Engine: Cells {(1, 4)}         | STANDING     | Split: False | Won: False | Lost: False
  => MATCH
Step 3: Action 'D'
  Replay: Cells {(2, 4), (3, 4)} | HORIZONTAL_X | Split: False | Won: False | Lost: False
  Engine: Cells {(2, 4), (3, 4)} | HORIZONTAL_X | Split: False | Won: False | Lost: False
  => MATCH
... (steps 4-6 omitted) ...
Step 7: Action 'D'
  Replay: Cells {(4, 7)}         | STANDING     | Split: False | Won: True  | Lost: False
  Engine: Cells {(4, 7)}         | STANDING     | Split: False | Won: True  | Lost: False
  => MATCH | VERDICT: AC
[TRACE END] 3D board triggers Victory Hole Descent and Confetti FX
```

D Video demo và mã nguồn

- Video demo Bloxorz.
- Mã nguồn trên GitHub.

E Ghi nhận hỗ trợ AI

Codex hỗ trợ tổ chức nội dung, biên tập và rút gọn báo cáo, diễn giải toán học, đồng thời tổng hợp dữ liệu từ CSV thực nghiệm được cung cấp. Ví dụ prompt đại diện (rút gọn): "Tạo các bảng median (IQR) theo stage từ CSV; chỉ dùng số liệu đã cung cấp, không thêm số liệu hoặc kết quả UCS." Nhóm chịu trách nhiệm rà soát, xác minh và quyết định nội dung cuối.

F Tài nguyên và thư viện

- Ursina 8.3.0[11].
- Panda3D 1.10.16[4].
- Pillow 13.0.0[5].